Data Structures and Algorithms -23CSE203

Lab Manual

Submitted By

NAME	K. Naveen
ROLL NO	AV.SC.U4CSE25122
YEAR/SEM/SECTION	2NDYEAR/ SEM-3/CSE-B
CREDITS	05

Submitted to

NAME	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR (Sl. Grade)
DEPARTMENT	SCHOOL OF COMPUTING

Index

S.NO	TITLE :	DATE
1	**WEEK 1:** **Recursion** A) Recursive program for countdown from n to 1, followed by 'launch'. B) Recursive program to calculate P^n (compound interest growth factor P, years n). C) Recursive program to print first n terms of Fibonacci series. D) Recursive program to search for an employee ID in a list. E) Recursive program to calculate factorial (n!) for parcel arrangement.	14-07-26
2	**WEEK 2:** **Searching** A) Python program to perform Linear Search on given input. B) Python program to perform Binary Search on given input (verify with sorted and unsorted lists).	21-07-26
3	**WEEK 3:** **Elementary Sorting** A) Python program to implement Bubble Sort (user input). B) Python program to implement Selection Sort (user input). C) Python program to implement Insertion Sort (user input).	01-08-26
4	**WEEK 4:** **Advanced Sorting** A) Python program to implement Merge Sort (user input). B) Python program to implement Quick Sort (user input).	11-08-26
5	**WEEK 5:** **Linked Lists** Implement operations for: A) Singly Linked List (SLL) B) Doubly Linked List (DLL) C) Circular Linked List (CLL) Operations: 1. Create LL 2. Insert at Beginning 3. Insert at End 4. Insert at Index 5. Delete by Value 6. Delete Prior Node 7. Delete Last Node 8. Count Nodes 9. Display/Traverse 10. Exit	18-08-26
6	**WEEK 6:** **Stacks** A) Implement Stack operations (Push, Pop, Peek, Display) using Array. B) Implement Stack operations (Push, Pop, Peek, Display) using Linked List.	25-08-26
7	**WEEK 7:** **Queues** A) Implement Queue using Arrays. B) Implement Queue using Linked List. C) Implement Circular Queue using Arrays. D) Implement Circular Queue using Linked List.	22-09-26
8	**WEEK 8:** **Binary Trees** A) Implement Binary Tree with operations using Array. B) Implement Binary Tree with operations using Linked List.	29-09-26
9	**WEEK 9:** **Binary Search Trees** Python program to implement Binary Search Tree (BST) operations.	06-10-26
10	**Week-10:** **AVL Tree**	13-10-26
11	**WEEK 11:** **Hashing** Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).	27-10-26

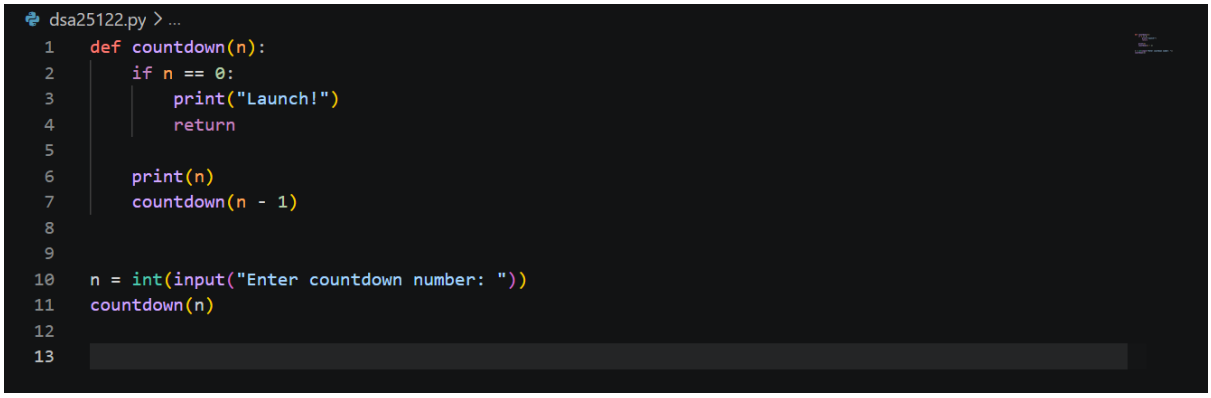
Lab-1

1a)

Aim:

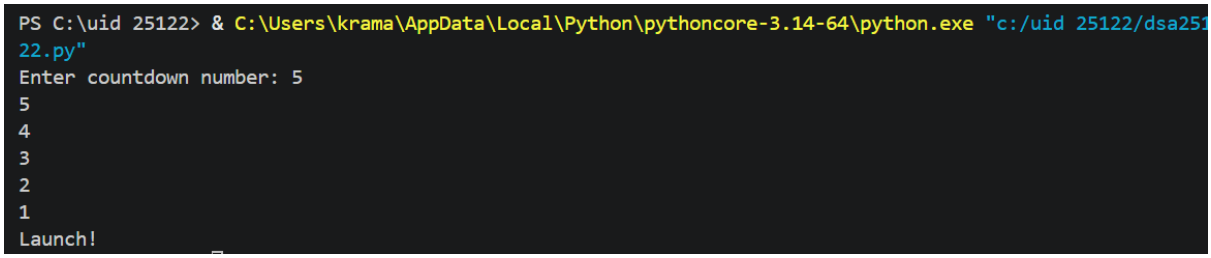
A rocket launch system displays a count down before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display count down from n to 1 followed by launch.

Program:



```
dsa25122.py > ...
1   def countdown(n):
2       if n == 0:
3           print("Launch!")
4           return
5
6       print(n)
7       countdown(n - 1)
8
9
10  n = int(input("Enter countdown number: "))
11  countdown(n)
12
13
```

Output:



```
PS C:\uid 25122> & C:\Users\krama\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/uid 25122/dsa251
22.py"
Enter countdown number: 5
5
4
3
2
1
Launch!
```

Error Table:

Error	Rectification

Description:

This program uses recursion to display a countdown from n to 1. The function calls itself with n - 1 until n becomes 0, then it displays "Launch".

1b)

Aim:

Banks Calculate the growth of investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n where

P = Principal growth amount, n = no of years

Program:

```python
def power(P, n):
    if n == 0:
        return 1
    return P * power(P, n - 1)


P = int(input("Enter P: "))
n = int(input("Enter number of years: "))

result = power(P, n)
print("P^n =", result)
```

Output:

```
PS C:\uid 25122> & C:\Users\krama\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/uid 25122/dsa25122.py"
Enter P: 2
Enter number of years: 5
P^n = 32
PS C:\uid 25122>
```

Error Table:

Error	Rectification
Negative values of n were not handled	Added n < 0 condition to handle negative number of years

Description:

The program recursively calculates P^n by multiplying P with the result of power(P, n-1). The recursion stops when n becomes 0.

1c)

Aim:

A HR department stores employee details in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

Program:

```python
def search_employee(emp_list, emp_id, index):
    if index == len(emp_list):
        return False

    if emp_list[index] == emp_id:
        return True

    return search_employee(emp_list, emp_id, index + 1)


emp_list = [101, 102, 103, 104, 105]

emp_id = int(input("Enter employee ID to search: "))

if search_employee(emp_list, emp_id, 0):
    print("Employee ID found")
else:
    print("Employee ID not found")
```

Output:

```
PS C:\uid 25122> & C:\Users\krama\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/uid 25122/dsa251
22.py"
Enter employee ID to search: 103
Employee ID found
PS C:\uid 25122>
```

Error Table:

Error	Rectification
Employee ID not found in the list	Added a condition to return "Employee ID not found"

Description:

The program recursively searches for a given employee ID in a list. It compares the employee ID with each element and continues searching until the ID is found or the end of the list is reached.

1d)

Aim:

In a logistic department, the no of possible ways to arrange n diff parcels are calculated using the factorial formula(n!). Write a recursive formula to find the factorial of a given number

Program:

```python
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)


n = int(input("Enter a number: "))

if n < 0:
    print("Factorial is not defined for negative numbers")
else:
    print("Factorial of", n, "is", factorial(n))
```

Output:

```
Enter a number: 55
Factorial of 55 is 12696403353658275925965100847566516959580321051449436762275840000000000000
PS C:\uid 25122>
```

Error Table:

Error	Rectification
Negative values were not handled	Added if n < 0 condition

Description:

The program uses recursion to calculate the factorial of a given number. The function repeatedly multiplies n by the factorial of n-1 until it reaches the base case 0 or 1.

1e)

Aim:

Print first n terms in a fibanocci number.

Program:

dsa25122.py > ...

C:\uid 25122\dsa25122.py • Untracked

```
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)


n = int(input("Enter the number of terms: "))

if n <= 0:
    print("Enter a positive number")
else:
    for i in range(n):
        print(fibonacci(i), end=" ")
```

Output:

```
Enter the number of terms: 5
0 1 1 2 3
PS C:\uid 25122>
```

Error Table:

Error	Rectification
Zero or negative number of terms was not handled	Added if n <= 0 condition

Description:

The program uses recursion to generate the Fibonacci series. Each term is calculated by adding the previous two terms, starting with 0 and 1.

Lab – 2

2a)

Aim:

Write a program for the linear search algorithm

Program:

```python
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1


arr = list(map(int, input("Enter the elements: ").split()))
key = int(input("Enter the element to search: "))

result = linear_search(arr, key)

if result != -1:
    print("Element found at position", result + 1)
else:
    print("Element not found")
```

Output:

```
Enter the elements: 1 2 4 5
Enter the element to search: 4
Element found at position 3
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122>
```

Error Table:

Error	Rectification
Element not found was not handled	Added return -1

Description:

The program searches for an element by checking each element of the list sequentially until the element is found or the end of the list is reached.

2b)

Aim:

Write a program for binary search algorithm considering the elements to be sorted

Program:

```
week-2 > 2(b).py > ...
1   def binary_search(arr, key):
2       low = 0
3       high = len(arr) - 1
4
5       while low <= high:
6           mid = (low + high) // 2
7
8           if arr[mid] == key:
9               return mid
10          elif arr[mid] < key:
11              low = mid + 1
12          else:
13              high = mid - 1
14
15      return -1
16
17
18  arr = list(map(int, input("Enter sorted elements: ").split()))
19  key = int(input("Enter the element to search: "))
20
21  result = binary_search(arr, key)
22
23  if result != -1:
24      print("Element found at position", result + 1)
25  else:
26      print("Element not found")
        Debug Console (Ctrl+Shift+Y)
```

Output:

```
Enter sorted elements: 4 5 8 3 0 9
Enter the element to search: 5
Element found at position 2
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122>
```

Error Table:

Error	Rectification
Wrote the mid formula outside the while loop which gave the infinite loop error	Added the mid formula inside the while loop

Description:

The program implements Binary Search by repeatedly dividing a sorted list into two halves and searching only the half that may contain the required element.

2c)

Aim:

Write a program for implementation of binary search along with the sorting the elements

Program:

```python
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1

arr = list(map(int, input("Enter the elements: ").split()))
arr.sort()
print("Sorted elements:", arr)
key = int(input("Enter the element to search: "))
result = binary_search(arr, key)

if result != -1:
    print("Element found at position", result + 1)
else:
    print("Element not found")
```

Output:

```
Enter the elements: 4 7 3 9 8 2 1 5
Sorted elements: [1, 2, 3, 4, 5, 7, 8, 9]
Enter the element to search: 9
Element found at position 8
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122>
```

Error Table:

Error	Rectification
Used sorted method instead of which need to be referred to another variable as it returns a sorted array	Used the arr.sort() method which directly sorts the arr without using another variable

Description:

The program first sorts the input elements using arr.sort(). Then, it uses the Binary Search algorithm to find the required element by repeatedly dividing the sorted list into two halves.

Lab – 3

3a)

Aim:

Write a python program to implement bubble sort where the input elements are taken from the user.

Program:

```
week-2 > 2(c).py > ...
arr = list(map(int, input("Enter the elements: ").split()))

n = len(arr)

for i in range(n):
    for j in range(0, n - i - 1):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]

print("Sorted elements:", arr)
```

Output:

```
Enter the elements: 4 6 3 1 9 7 5
Sorted elements: [1, 3, 4, 5, 6, 7, 9]
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122>
```

Error Table:

Error	Rectification
Gave the numbers in descending order	Changed the sign symbol while comparing the adjacent elements.

Description:

The program implements Bubble Sort by comparing adjacent elements and swapping them when they are in the wrong order. This process is repeated until all elements are arranged in ascending order.

3b)

Aim:

Write a python program to implement insertion sort where the input elements are taken from the user.

Program:

```
Click to add a breakpoint int, input("Enter the elements: ").split()))

for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1

    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1

    arr[j + 1] = key

print("Sorted elements:", arr)
```

Output:

```
week-3\3(b).py"
Enter the elements: 6 7 8 4 3 9 1
Sorted elements: [1, 3, 4, 6, 7, 8, 9]
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122>
```

Error Table:

Error	Rectification

Description:

The program implements Insertion Sort by taking one element at a time and inserting it into its correct position among the already sorted elements.

3c)

Aim:

Write a python program to implement selection sort where the input elements are taken from the user.

Program:

```
arr = list(map(int, input("Enter the elements: ").split()))

n = len(arr)

for i in range(n):
    min_index = i

    for j in range(i + 1, n):
        if arr[j] < arr[min_index]:
            min_index = j
            (variable) i: int
    arr[i], arr[min_index] = arr[min_index], arr[i]

print("Sorted elements:", arr)
```

Output:

```
week-3\3(c).py"
Enter the elements: 5 6 9 4 3 7
Sorted elements: [3, 4, 5, 6, 7, 9]
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122>
```

Error Table:

Error	Rectification

Description:

The program implements Selection Sort by finding the smallest element from the unsorted part of the list and swapping it with the first element of that part. This process continues until all elements are sorted in ascending order.

Lab-4

4a)

Aim:

Write a python program to implement merge sort where the input elements are taken from the user

Program:



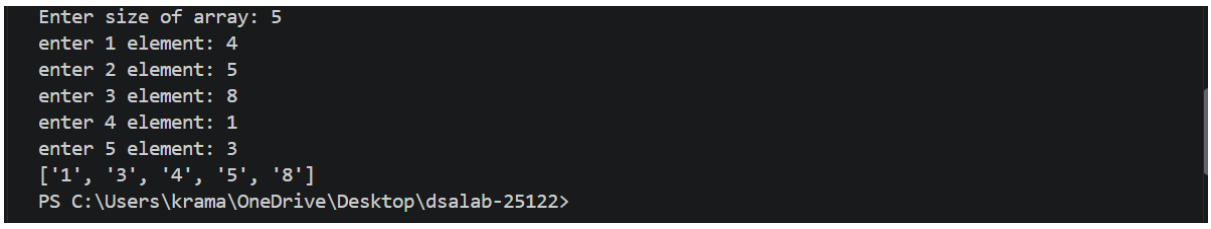
```
week-4 > 4(a).py > ...
def main():
    n = int(input("Enter size of array: "))
    arr = []
    for i in range(n):
        arr.append(input(f"enter {i+1} element: "))
    sorted_arr = merge_sort(arr)
    print(sorted_arr)

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    arr = []
    i = 0
    j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j] :
            arr.append(left[i])
            i += 1
        else:
            arr.append(right[j])
            j += 1
    arr.extend(left[i:])
    arr.extend(right[j:])
    return arr

if __name__ == '__main__':
    main()
```

Output:



```
Enter size of array: 5
enter 1 element: 4
enter 2 element: 5
enter 3 element: 8
enter 4 element: 1
enter 5 element: 3
['1', '3', '4', '5', '8']
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122>
```

Error Table:

Error	Rectification

RecursionError: maximum recursion depth exceeded	base case in merge_sort() len(arr) <1 to len(arr) <= 1
PS C:\Users\Lab\Documents\cse25142\dsa-lab\week-4> python merge_sort.py [2, 3, 7, 7, 8, 3, 7, 7, 8, 7, 7, 8]	kept the extend in the while loop instead of out of while loop

Description:

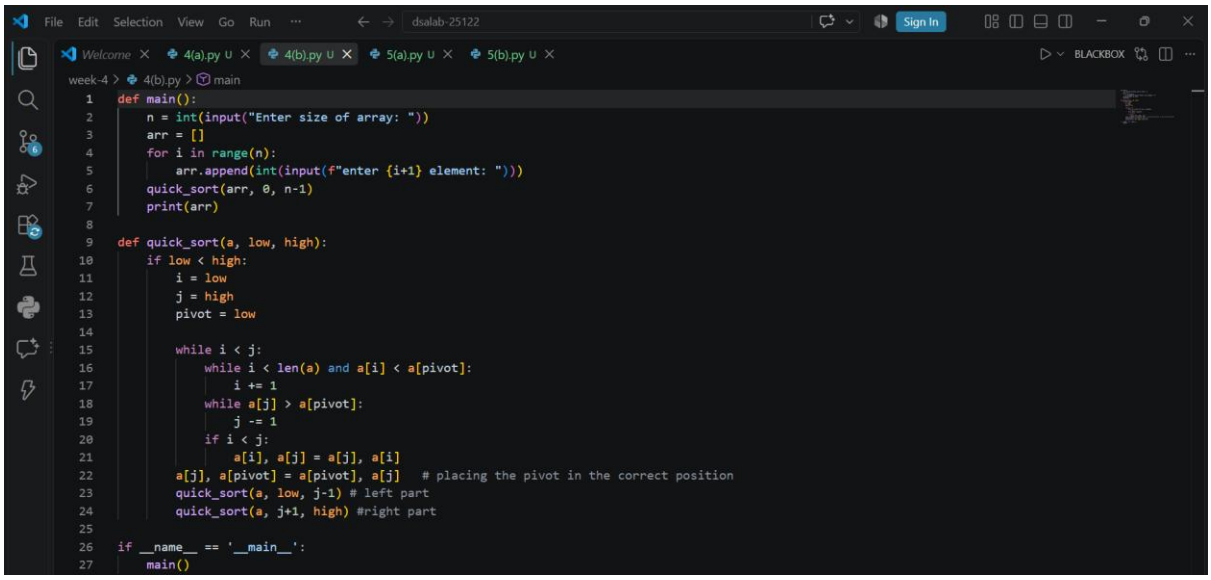
This Python program implements the Merge Sort algorithm to sort the elements of an array in ascending order. The program first takes the size and elements of the array from the user through the main() function. The merge_sort() function recursively divides the array into two halves until each sub-array contains one or zero elements. The merge() function then compares the elements of the two sorted halves and combines them into a single sorted array. The append() function adds elements to the result array, while the extend() function adds any remaining elements. Finally, the sorted array is displayed to the user. Merge Sort follows the divide-and-conquer approach and has a time complexity of O(n log n).

4b)

Aim:

Write a python program to implement quick sort where the input elements are taken from the user.

Program:



```
def main():
    n = int(input("Enter size of array: "))
    arr = []
    for i in range(n):
        arr.append(int(input(f"enter {i+1} element: ")))
    quick_sort(arr, 0, n-1)
    print(arr)

def quick_sort(a, low, high):
    if low < high:
        i = low
        j = high
        pivot = low

        while i < j:
            while i < len(a) and a[i] < a[pivot]:
                i += 1
            while a[j] > a[pivot]:
                j -= 1
            if i < j:
                a[i], a[j] = a[j], a[i]
        a[j], a[pivot] = a[pivot], a[j]   # placing the pivot in the correct position
        quick_sort(a, low, j-1) # left part
        quick_sort(a, j+1, high) #right part

if __name__ == '__main__':
    main()
```

Output:

```
week-4\4(b).py"
Enter size of array: 4
enter 1 element: 9
enter 2 element: 3
enter 3 element: 6
enter 4 element: 4
[3, 4, 6, 9]
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122>
```

Error Table:

PS C:\Users\Lab\Documents\cse25122\dsa-lab-25122\week-4> python quick_sort.py Enter size of array: 7 enter 1 element: 2 enter 2 element: 3 enter 3 element: 56 enter 4 element: 9 enter 5 element: -` enter 6 element:	Instead of inputting the integers, inputted the string added int before the input arr.append(int(input(f”enter {i+1} element”)

enter 7 element: 2	
PS C:\Users\Lab\Documents\cse25122\dsa-lab-25122\week-4> python quick_sort.py Traceback (most recent call last): File "C:\Users\Lab\Documents\cse25122\dsa-lab\week-4\quick_sort.py", line 24, in <module> main() File "C:\Users\Lab\Documents\2 \dsa-lab25122\week-4\quick_sort.py", line 5, in main quick_sort(arr) TypeError: quick_sort() missing 2 required positional arguments: 'low' and 'high' PS C:\Users\Lab\Documents\cse25142\dsa-lab\week-4>	added the two parameters to the function
Traceback (most recent call last): File "C:\Users\Lab\Documents\cse25122\dsa-lab-25122\week-4\quick_sort.py", line 29, in <module> main() File "C:\Users\Lab\Documents\cse25142\dsa-lab\week-4\quick_sort.py", line 9, in main print(quick_sort(arr, 0, n-1)) ^^^^^^^^^^^^^^^^^^^^^^^ File "C:\Users\Lab\Documents\cse25122\dsa-lab25122\week-4\quick_sort.py", line 26, in quick_sort quick_sort(a, j+1, high) #right part ^^^^^^^^^^^^^^^^^^^^^^^^	Placed a comma

File "C:\Users\Lab\Documents\cse25122\dsa-lab-25122\week-4\quick_sort.py", line 24, in quick_sort a[j], a[pivot] = a[pivot] = a[j] # placing the pivot in the correct position ^^^^^^^^^^^^^^ ValueError: too many values to unpack (expected 2)	

Description:

This Python program implements the Quick Sort algorithm to sort the elements of an array in ascending order. The program first takes the size and elements of the array from the user. The quick_sort() function selects the first element as the pivot and uses two pointers, i and j, to compare and rearrange the elements around the pivot. Elements smaller than the pivot are moved to the left, while elements greater than the pivot are moved to the right. After placing the pivot in its correct position, the function is recursively called for the left and right portions of the array. Finally, the sorted array is displayed. Quick Sort follows the divide-and-conquer approach and has an average time complexity of O(n log n).

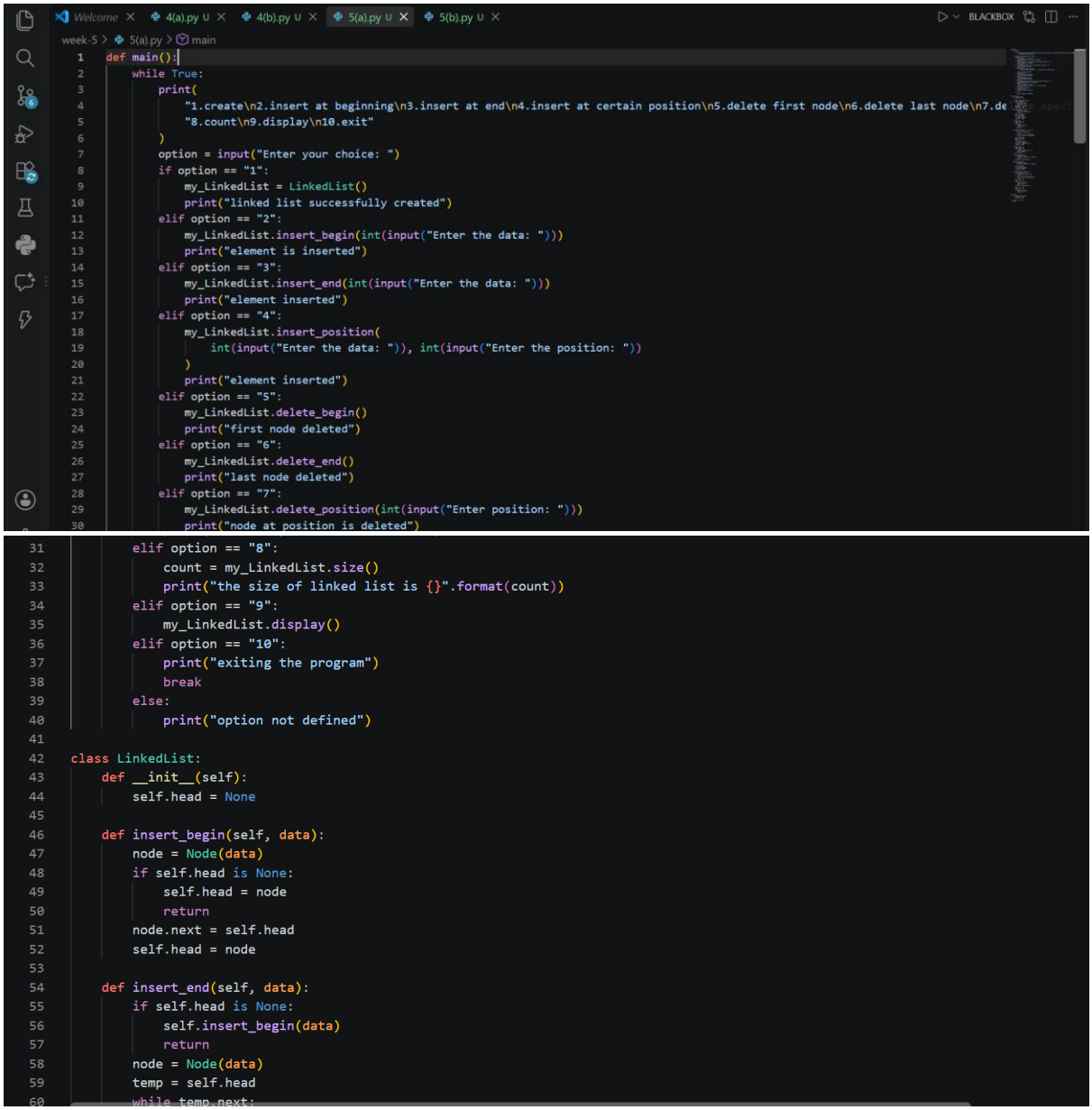
Week-5

5(a)Aim:

Write a Python program to implement a Singly Linked List (SLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit

Program:



```python
def main():
    while True:
        print(
            "1.create\n2.insert at beginning\n3.insert at end\n4.insert at certain position\n5.delete first node\n6.delete last node\n7.de
            "8.count\n9.display\n10.exit"
        )
        option = input("Enter your choice: ")
        if option == "1":
            my_LinkedList = LinkedList()
            print("linked list successfully created")
        elif option == "2":
            my_LinkedList.insert_begin(int(input("Enter the data: ")))
            print("element is inserted")
        elif option == "3":
            my_LinkedList.insert_end(int(input("Enter the data: ")))
            print("element inserted")
        elif option == "4":
            my_LinkedList.insert_position(
                int(input("Enter the data: ")), int(input("Enter the position: "))
            )
            print("element inserted")
        elif option == "5":
            my_LinkedList.delete_begin()
            print("first node deleted")
        elif option == "6":
            my_LinkedList.delete_end()
            print("last node deleted")
        elif option == "7":
            my_LinkedList.delete_position(int(input("Enter position: ")))
            print("node at position is deleted")
        elif option == "8":
            count = my_LinkedList.size()
            print("the size of linked list is {}".format(count))
        elif option == "9":
            my_LinkedList.display()
        elif option == "10":
            print("exiting the program")
            break
        else:
            print("option not defined")

class LinkedList:
    def __init__(self):
        self.head = None

    def insert_begin(self, data):
        node = Node(data)
        if self.head is None:
            self.head = node
            return
        node.next = self.head
        self.head = node

    def insert_end(self, data):
        if self.head is None:
            self.insert_begin(data)
            return
        node = Node(data)
        temp = self.head
        while temp.next:
```

```
            temp = temp.next
        temp.next = node

    def size(self):
        temp = self.head
        count = 0
        while temp:
            temp = temp.next
            count += 1
        return count

    def insert_position(self, data, pos):
        if pos - 1 == 0:
            self.insert_begin(data)
            return
        if pos < 1 or pos > self.count():
            print("position not available")
            return
        temp = self.head
        node = Node(data)
        count = 0
        while count < pos - 1:
            temp = temp.next
        node.next = temp.next.next
        temp.next = node

    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("None")

    def delete_begin(self):
        if self.head is None:
            print("The Linked list is empty")
            return
        self.head = self.head.next

    def delete_end(self):
        if self.head is None:
            print("The Linked list is empty")
            return
        temp = self.head
        while temp.next.next:
            temp = temp.next
        temp.next = None

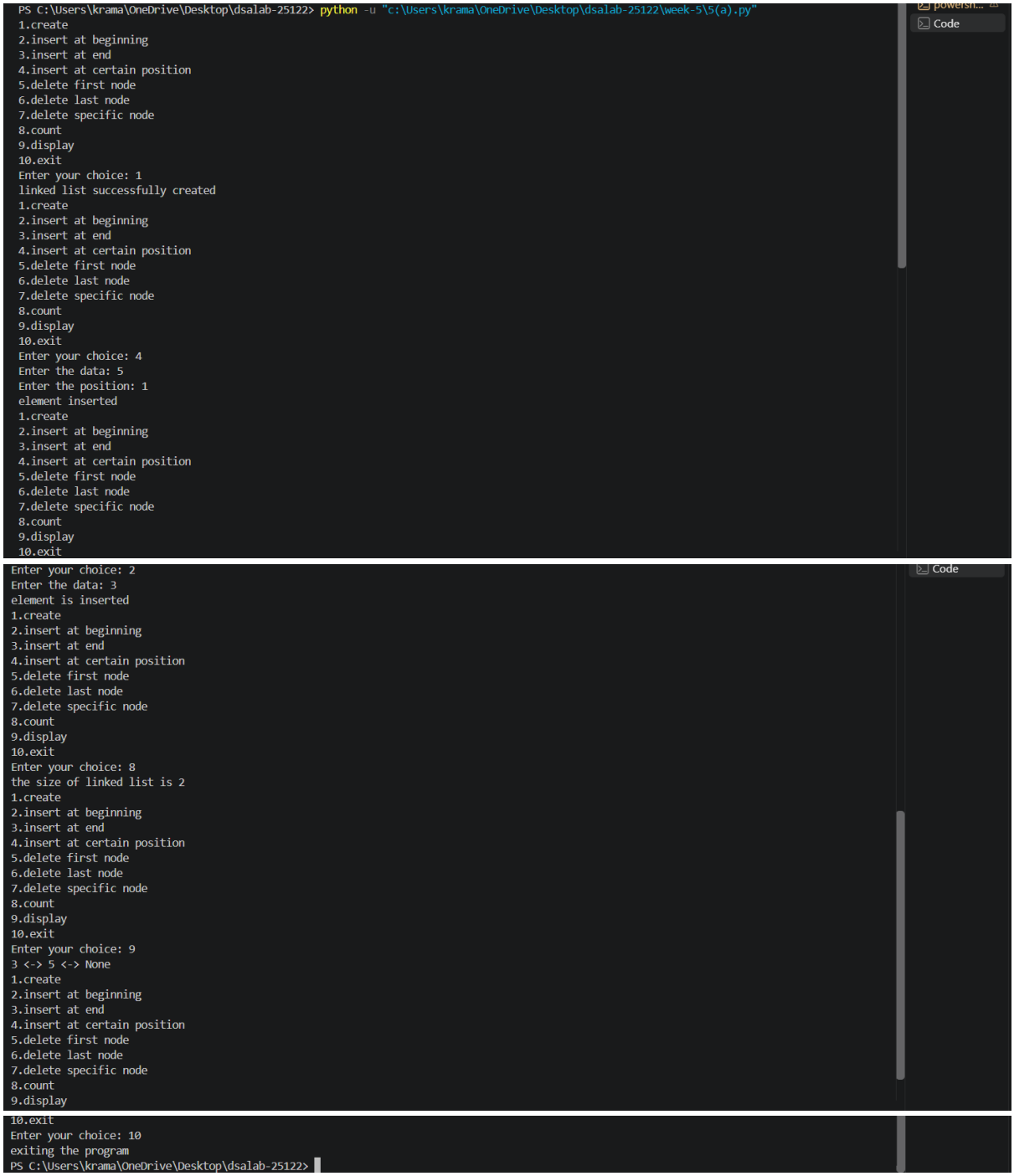
    def delete_position(self, pos):
        if self.head is None:
            print("Empty linked list")
            return
        if pos < 0 or pos > self.size():
            print("position do not exists")
            return
        temp = self.head
        count = 0
        while count < pos - 1:
            temp = temp.next
            count += 1
        temp.next = temp.next.next

        temp = self.head
        count = 0
        while count < pos - 1:
            temp = temp.next
            count += 1

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

if __name__ == "__main__":
    main()
```

Output:



Error Table:

Error	Rectification
The program may display an incorrect message for insertion at a specific index even when the index is invalid	Check whether the index is valid before displaying the successful insertion message.

Description:

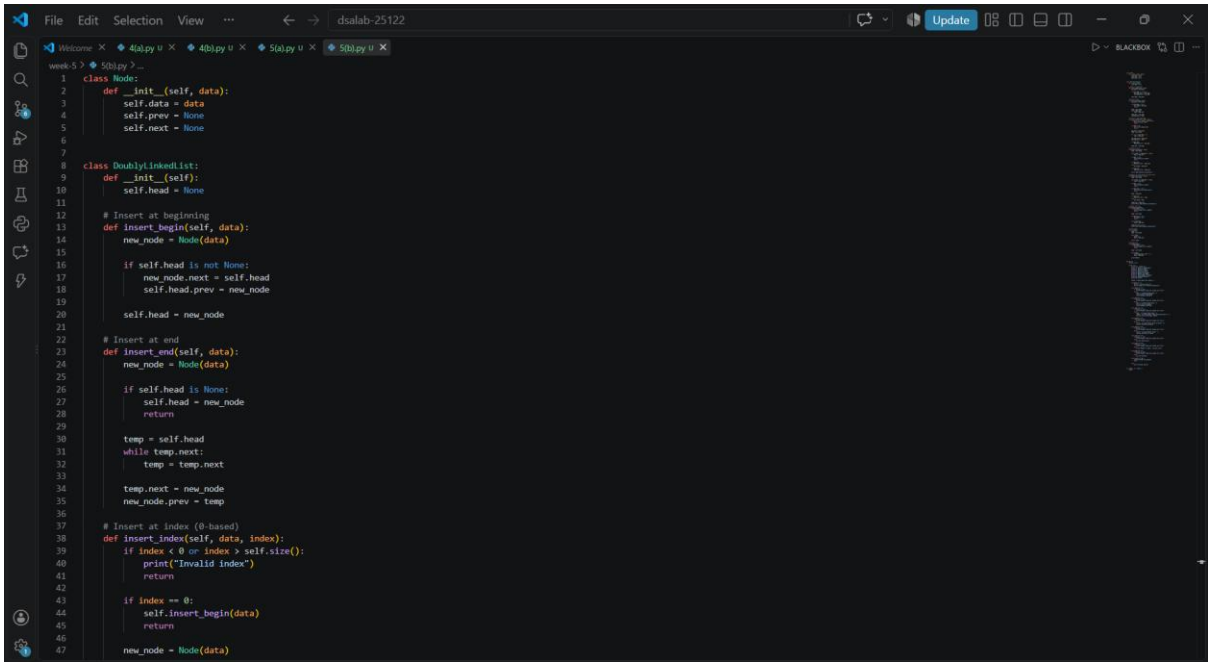
This Python program implements a Singly Linked List (SLL) using classes and provides a menu-driven interface to perform various operations. The program allows the user to create a linked list, insert nodes at the beginning, end, or a specified position, and delete nodes from the beginning, end, or a particular position. It also provides operations to count the total number of nodes and display all the elements in the linked list. The Node class is used to store data and maintain a reference to the next node, while the LinkedList class contains methods to perform the different operations. The main() function displays a menu and accepts the user's choice to execute the corresponding operation. The program continues running until the user selects the exit option.

5(b)Aim:

Write a Python program to implement a Double Linked List (DLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit

Program:



```python
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None

    # Insert at beginning
    def insert_begin(self, data):
        new_node = Node(data)

        if self.head is not None:
            new_node.next = self.head
            self.head.prev = new_node

        self.head = new_node

    # Insert at end
    def insert_end(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

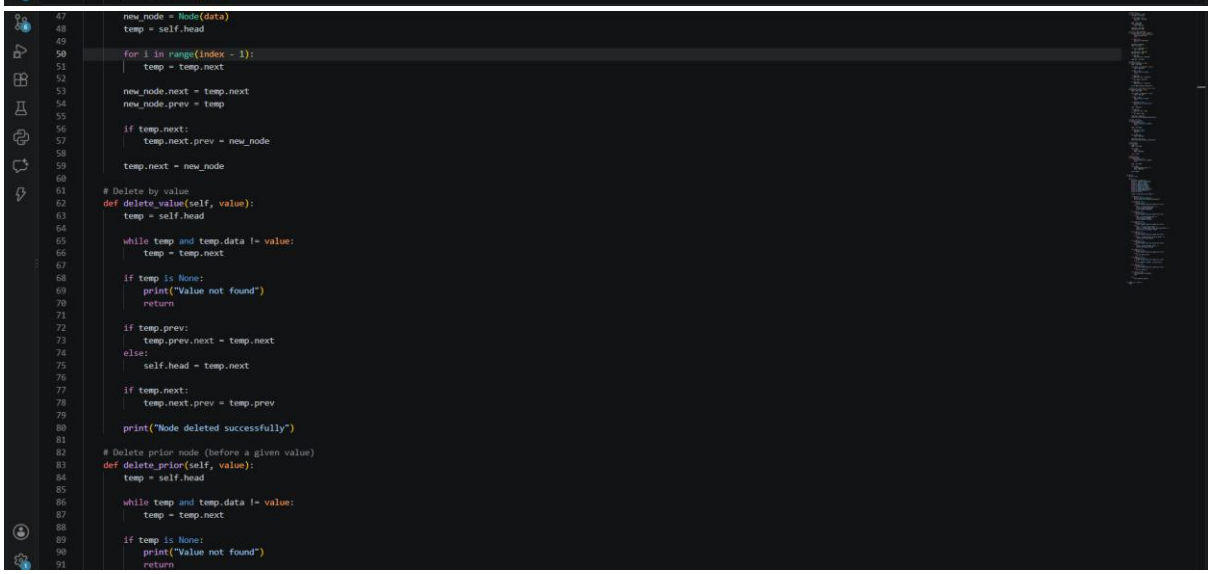
        temp = self.head
        while temp.next:
            temp = temp.next

        temp.next = new_node
        new_node.prev = temp

    # Insert at index (0-based)
    def insert_index(self, data, index):
        if index < 0 or index > self.size():
            print("Invalid index")
            return

        if index == 0:
            self.insert_begin(data)
            return

        new_node = Node(data)
```



```python
        new_node = Node(data)
        temp = self.head

        for i in range(index - 1):
            temp = temp.next

        new_node.next = temp.next
        new_node.prev = temp

        if temp.next:
            temp.next.prev = new_node

        temp.next = new_node

    # Delete by value
    def delete_value(self, value):
        temp = self.head

        while temp and temp.data != value:
            temp = temp.next

        if temp is None:
            print("Value not found")
            return

        if temp.prev:
            temp.prev.next = temp.next
        else:
            self.head = temp.next

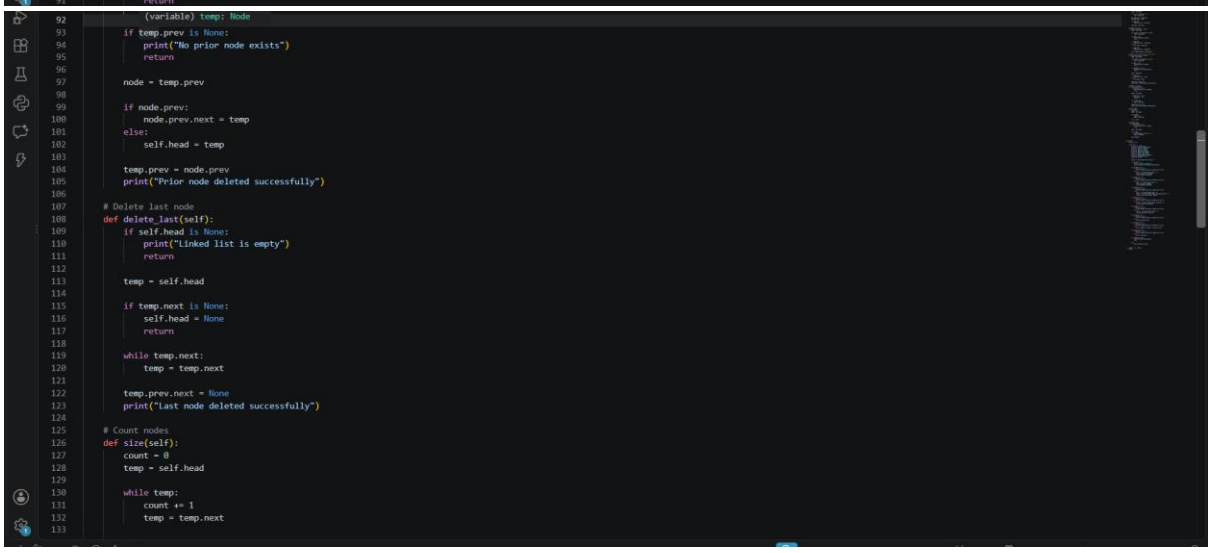
        if temp.next:
            temp.next.prev = temp.prev

        print("Node deleted successfully")

    # Delete prior node (before a given value)
    def delete_prior(self, value):
        temp = self.head

        while temp and temp.data != value:
            temp = temp.next

        if temp is None:
            print("Value not found")
            return
```



```python
            (variable) temp: Node
        if temp.prev is None:
            print("No prior node exists")
            return

        node = temp.prev

        if node.prev:
            node.prev.next = temp
        else:
            self.head = temp

        temp.prev = node.prev
        print("Prior node deleted successfully")

    # Delete last node
    def delete_last(self):
        if self.head is None:
            print("Linked list is empty")
            return

        temp = self.head

        if temp.next is None:
            self.head = None
            return

        while temp.next:
            temp = temp.next

        temp.prev.next = None
        print("Last node deleted successfully")

    # Count nodes
    def size(self):
        count = 0
        temp = self.head

        while temp:
            count += 1
            temp = temp.next
```



```
        return count

    # Display list
    def display(self):
        if self.head is None:
            print("Linked list is empty")
            return

        temp = self.head

        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next

        print("None")


def main():
    my_list = None

    while True:
        print("\n1. Create LL")
        print("2. Insert at Beginning")
        print("3. Insert at End")
        print("4. Insert at Index")
        print("5. Delete by Value")
        print("6. Delete Prior Node")
        print("7. Delete Last Node")
        print("8. Count Number of Nodes")
        print("9. Display/Traverse")
        print("10. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            my_list = DoublyLinkedList()
            print("Linked list created successfully")

        elif choice == "2":
            if my_list is None:
                print("Please create the linked list first")
            else:
                data = int(input("Enter data: "))
                my_list.insert_begin(data)
                print("Element inserted")
```



```
        elif choice == "3":
            if my_list is None:
                print("Please create the linked list first")
            else:
                data = int(input("Enter data: "))
                my_list.insert_end(data)
                print("Element inserted")

        elif choice == "4":
            if my_list is None:
                print("Please create the linked list first")
            else:
                data = int(input("Enter data: "))
                index = int(input("Enter index (starting from 0): "))
                my_list.insert_index(data, index)

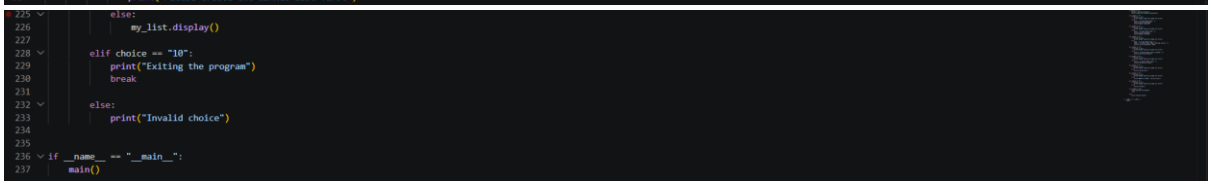
        elif choice == "5":
            if my_list is None:
                print("Please create the linked list first")
            else:
                value = int(input("Enter value to delete: "))
                my_list.delete_value(value)

        elif choice == "6":
            if my_list is None:
                print("Please create the linked list first")
            else:
                value = int(input("Enter value: "))
                my_list.delete_prior(value)

        elif choice == "7":
            if my_list is None:
                print("Please create the linked list first")
            else:
                my_list.delete_last()

        elif choice == "8":
            if my_list is None:
                print("Please create the linked list first")
            else:
                print("Number of nodes:", my_list.size())

        elif choice == "9":
            if my_list is None:
                print("Please create the linked list first")
```



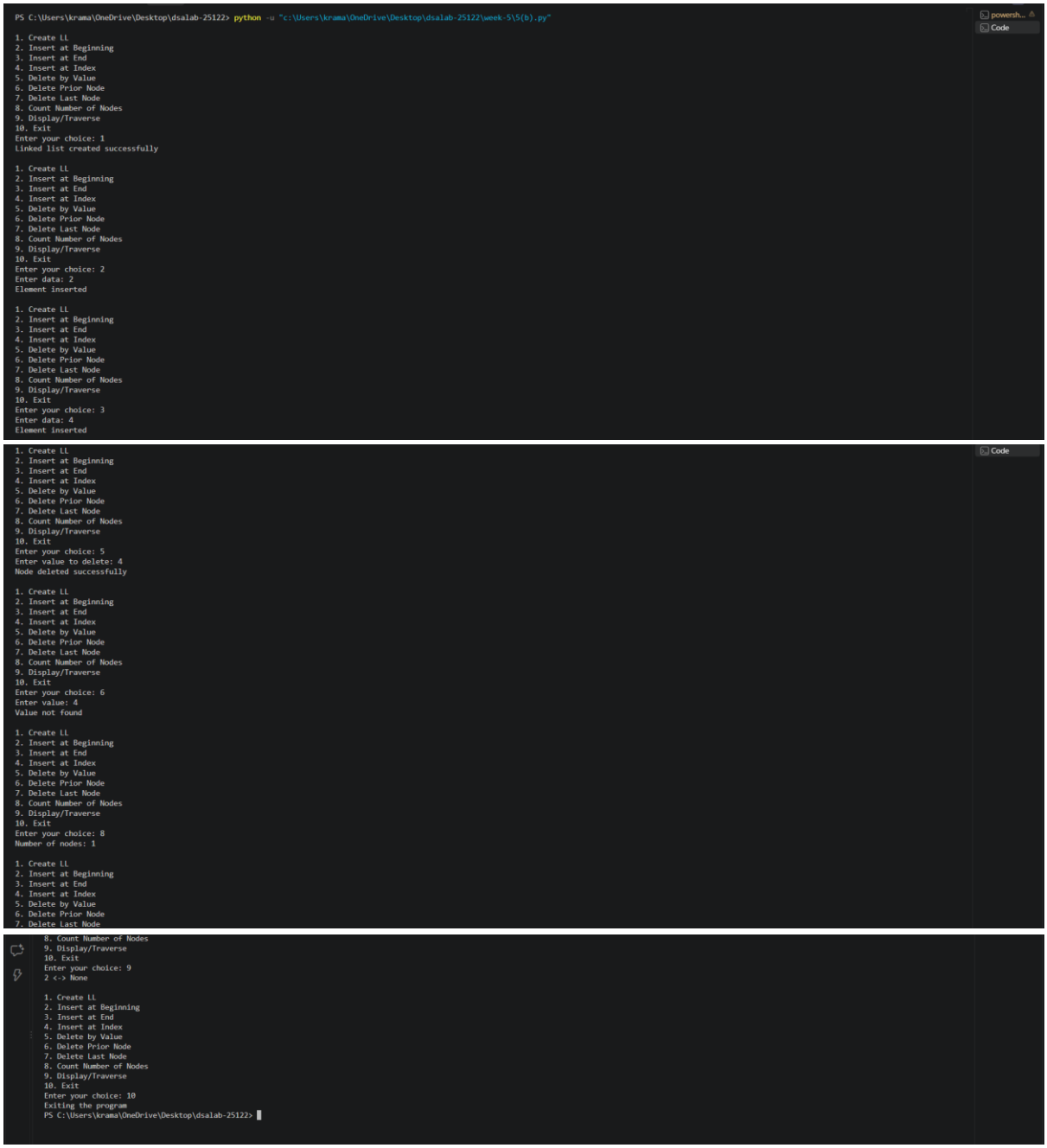
```
            else:
                my_list.display()

        elif choice == "10":
            print("Exiting the program")
            break

        else:
            print("Invalid choice")


if __name__ == "__main__":
    main()
```

Output:



Error Table:

Error	Rectification
The prev and next links may not be updated correctly during insertion or deletion.	Update both links whenever a node is inserted or deleted.

Description:

This Python program implements a Doubly Linked List (DLL) using classes and provides a menu-driven interface to perform various operations. The program allows the user to create a doubly linked list, insert nodes at the beginning, end, or a specified index, delete a node by value, delete the node prior to a specified value, and delete the last node. It also provides operations to count the total number of nodes and display all the elements in the linked list. The Node class stores data and two pointers, prev and next, which point to the previous and next nodes, respectively. The DoublyLinkedList class contains methods to perform all the operations. The main() function displays a menu and accepts the user's choice to execute the corresponding operation. The program continues running until the user selects the exit option.

5(c)Aim:

Write a Python program to implement a Circular Linked List (CLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit

Program:

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.head = None

    # Insert at beginning
    def insert_begin(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
            return

        temp = self.head
        while temp.next != self.head:
            temp = temp.next

        new_node.next = self.head
        temp.next = new_node
        self.head = new_node

    # Insert at end
    def insert_end(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
            return

        temp = self.head
        while temp.next != self.head:
            temp = temp.next

        temp.next = new_node
        new_node.next = self.head
```

```python
    def insert_index(self, data, index):
        if index < 0 or index > self.size():
            print("Invalid index")
            return

        if index == 0:
            self.insert_begin(data)
            return

        new_node = Node(data)
        temp = self.head

        for i in range(index - 1):
            temp = temp.next

        new_node.next = temp.next
        temp.next = new_node

    # Delete by value
    def delete_value(self, value):
        if self.head is None:
            print("Linked list is empty")
            return

        temp = self.head
        prev = None

        while True:
            if temp.data == value:
                break
            prev = temp
            temp = temp.next
            if temp == self.head:
                print("Value not found")
                return

        if temp == self.head:
            if self.head.next == self.head:
                self.head = None
                return
```

```python
            last = self.head
            while last.next != self.head:
                last = last.next

            self.head = self.head.next
            last.next = self.head
        else:
            prev.next = temp.next

        print("Node deleted successfully")

    # Delete prior node (before a given value)
    def delete_prior(self, value):
        if self.head is None or self.head.next == self.head:
            print("No prior node exists")
            return

        temp = self.head
        prev = None

        while temp.data != value:
            prev = temp
            temp = temp.next
            if temp == self.head:
                print("Value not found")
                return

        if temp == self.head:
            last = self.head
            while last.next.next != self.head:
                last = last.next

            last.next = self.head
            print("Prior node deleted successfully")
            return

        if temp == self.head.next:
            last = self.head
```

```
            self.head = temp
            last.next = self.head
            print("Prior node deleted successfully")
            return

        node = self.head
        while node.next != temp:
            node = node.next

        prev_node = self.head
        while prev_node.next != node:
            prev_node = prev_node.next

        prev_node.next = temp
        print("Prior node deleted successfully")

    # Delete last node
    def delete_last(self):
        if self.head is None:
            print("Linked list is empty")
            return

        if self.head.next == self.head:
            self.head = None
            print("Last node deleted successfully")
            return

        temp = self.head
        while temp.next.next != self.head:
            temp = temp.next

        temp.next = self.head
        print("Last node deleted successfully")
```

```
    # Count nodes
    def size(self):
        if self.head is None:
            return 0

        count = 1
        temp = self.head

        while temp.next != self.head:
            count += 1
            temp = temp.next

        return count

    # Display list
    def display(self):
        if self.head is None:
            print("Linked list is empty")
            return

        temp = self.head

        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
            if temp == self.head:
                break

        print("(Back to Head)")


def main():
    my_list = None

    while True:
        print("\n1. Create LL")
        print("2. Insert at Beginning")
        print("3. Insert at End")
```

```
        print("4. Insert at Index")
        print("5. Delete by Value")
        print("6. Delete Prior Node")
        print("7. Delete Last Node")
        print("8. Count Number of Nodes")
        print("9. Display/Traverse")
        print("10. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            my_list = CircularLinkedList()
            print("Linked list created successfully")

        elif choice == "2":
            if my_list is None:
                print("Please create the linked list first")
            else:
                data = int(input("Enter data: "))
                my_list.insert_begin(data)
                print("Element inserted")

        elif choice == "3":
            if my_list is None:
                print("Please create the linked list first")
            else:
                data = int(input("Enter data: "))
                my_list.insert_end(data)
                print("Element inserted")

        elif choice == "4":
            if my_list is None:
                print("Please create the linked list first")
            else:
                data = int(input("Enter data: "))
                index = int(input("Enter index (starting from 0): "))
                my_list.insert_index(data, index)

        elif choice == "5":
            if my_list is None:
                print("Please create the linked list first")
            else:
                value = int(input("Enter value to delete: "))
                my_list.delete_value(value)

        elif choice == "6":
            if my_list is None:
                print("Please create the linked list first")
            else:
                value = int(input("Enter value: "))
                my_list.delete_prior(value)

        elif choice == "7":
            if my_list is None:
                print("Please create the linked list first")
            else:
                my_list.delete_last()

        elif choice == "8":
            if my_list is None:
                print("Please create the linked list first")
            else:
                print("Number of nodes:", my_list.size())

        elif choice == "9":
            if my_list is None:
                print("Please create the linked list first")
            else:
                my_list.display()

        elif choice == "10":
            print("Exiting the program")
            break

        else:
            print("Invalid choice")


if __name__ == "__main__":
    main()
```

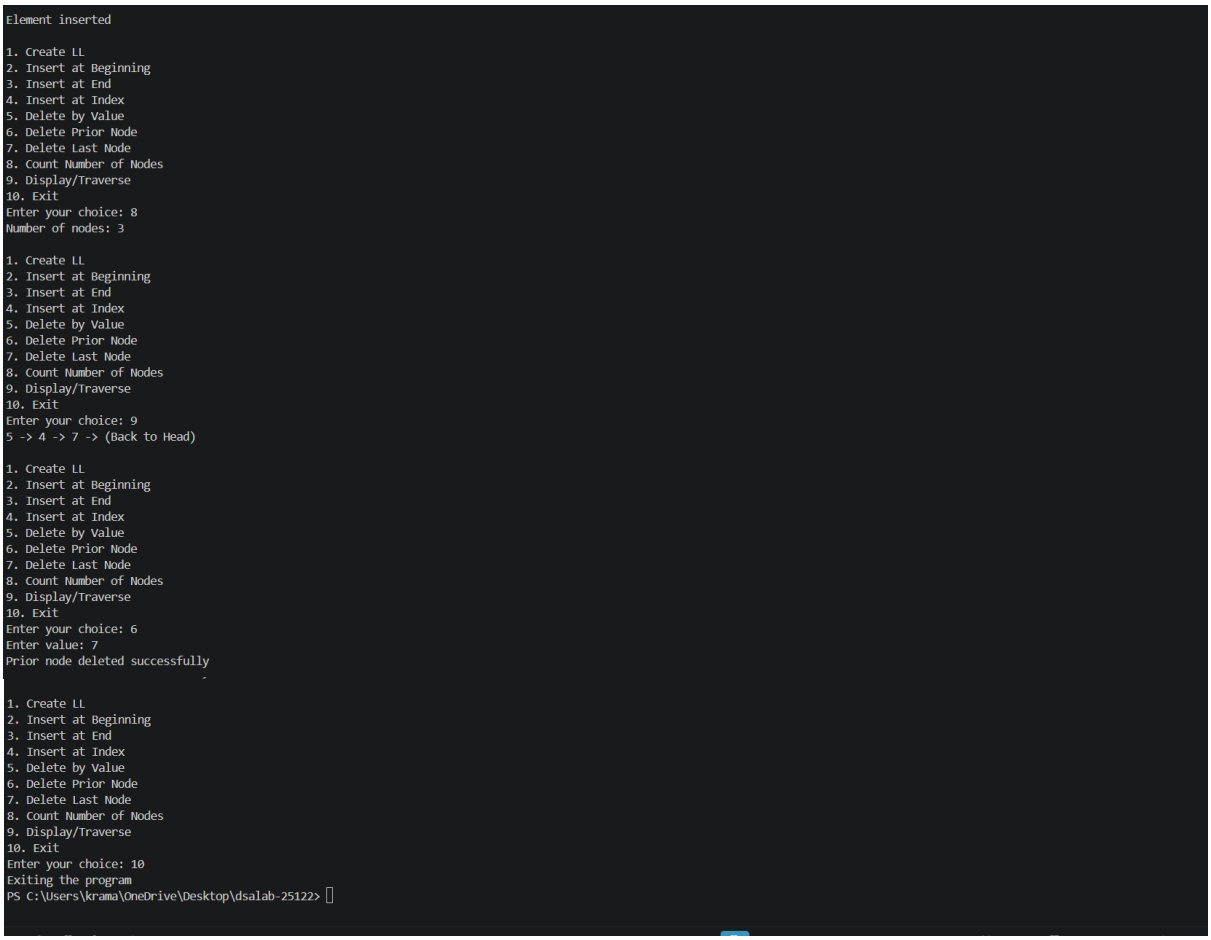
Output:

```
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122> python -u "c:\Users\krama\OneDrive\Desktop\dsalab-25122\week-5\5(c).py"

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit
Enter your choice: 1
Linked list created successfully

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit
Enter your choice: 2
Enter data: 4
Element inserted

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit
Enter your choice: 2
Enter data: 5
```



Error Table:

Error	Rectification
No errors	

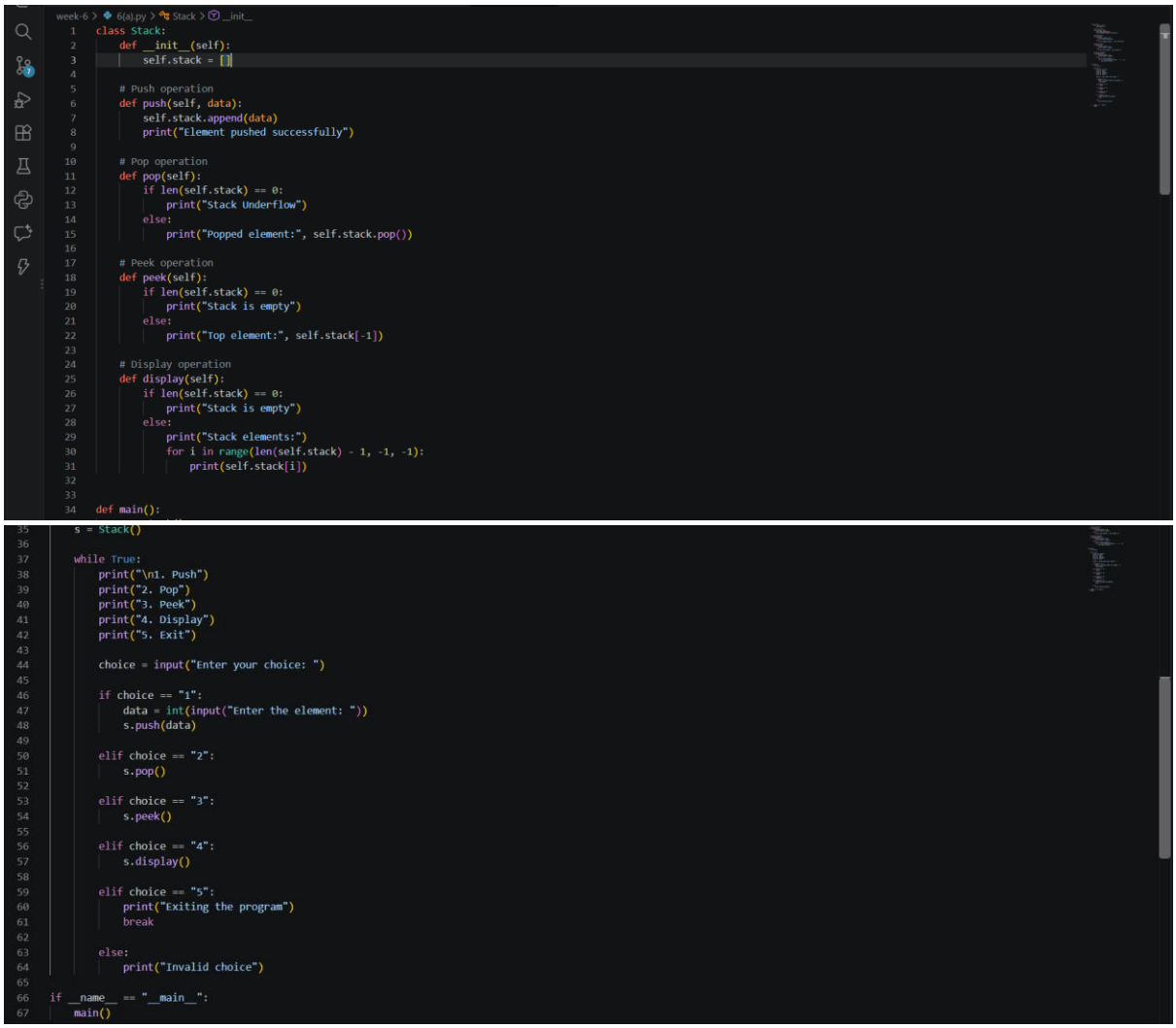
Description:

This Python program implements a Circular Linked List (CLL) using classes and provides a menu-driven interface to perform various operations. The program allows the user to create a circular linked list, insert nodes at the beginning, end, or a specified index, delete a node by value, delete the node prior to a specified value, and delete the last node. It also provides operations to count the total number of nodes and display all the elements in the linked list. The Node class is used to store data and maintain a reference to the next node, while the CircularLinkedList class contains methods to perform all the operations. In a circular linked list, the last node points back to the head instead of pointing to None. The main() function displays a menu and accepts the user's choice to execute the corresponding operation. The program continues running until the user selects the exit option.

Week-6

6(a)Aim:

Program:



```
week-6 > 6(a).py > Stack > __init__
class Stack:
    def __init__(self):
        self.stack = []

    # Push operation
    def push(self, data):
        self.stack.append(data)
        print("Element pushed successfully")

    # Pop operation
    def pop(self):
        if len(self.stack) == 0:
            print("Stack Underflow")
        else:
            print("Popped element:", self.stack.pop())

    # Peek operation
    def peek(self):
        if len(self.stack) == 0:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[-1])

    # Display operation
    def display(self):
        if len(self.stack) == 0:
            print("Stack is empty")
        else:
            print("Stack elements:")
            for i in range(len(self.stack) - 1, -1, -1):
                print(self.stack[i])


def main():
    s = Stack()

    while True:
        print("\n1. Push")
        print("2. Pop")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            data = int(input("Enter the element: "))
            s.push(data)

        elif choice == "2":
            s.pop()

        elif choice == "3":
            s.peek()

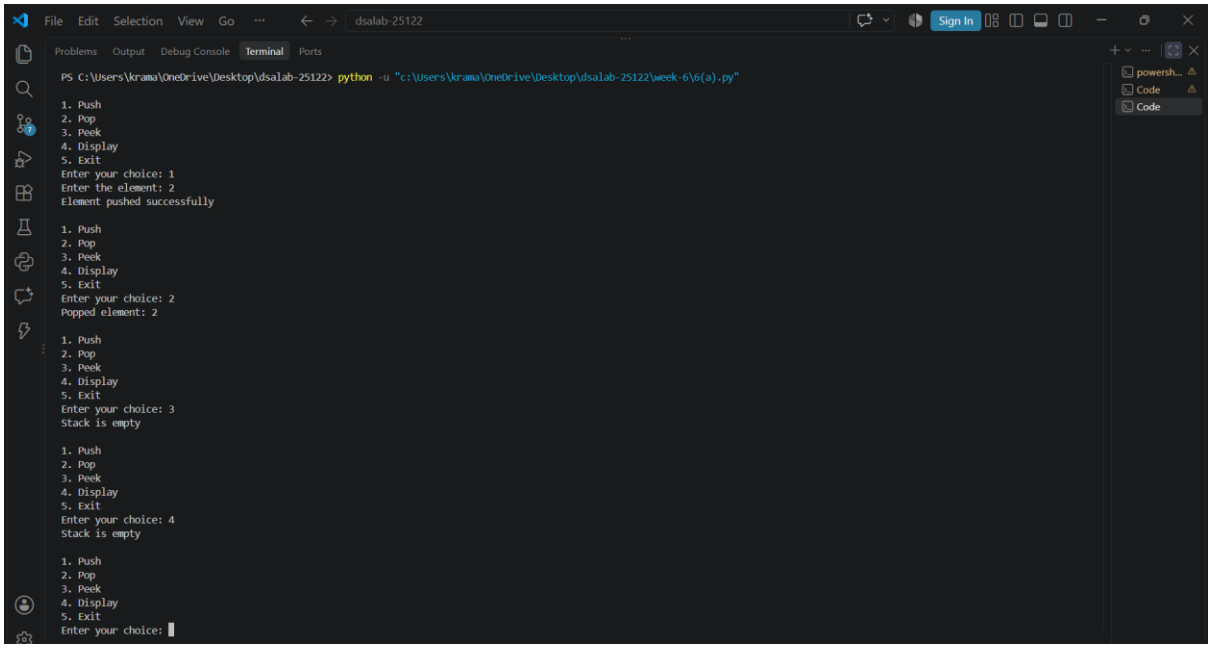
        elif choice == "4":
            s.display()

        elif choice == "5":
            print("Exiting the program")
            break

        else:
            print("Invalid choice")

if __name__ == "__main__":
    main()
```

Output:



Error Table:

Error	Rectification
In peek, self.peek is taken in ‘if’ does not give correct answer for stack underflow.	Self.top is taken in ‘if’ to give correct output.

Description:

This Python program implements a stack using an array (Python list) and provides a menu-driven interface to perform various operations. The program supports four main operations: Push, Pop, Peek, and Display. The Push operation inserts an element at the top of the stack, while the Pop operation removes the top element from the stack. The Peek operation displays the top element without removing it, and the Display operation shows all the elements in the stack from top to bottom. The Stack class contains methods to perform these operations, while the main() function displays a menu and accepts the user's choice to execute the corresponding operation. The program follows the LIFO (Last In, First Out) principle, where the last element inserted is the first one to be removed. It also handles stack underflow when a Pop operation is performed on an empty stack. The program continues running until the user selects the Exit option.

6(b)Aim:

Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using an Linked List

Program:

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack:
    def __init__(self):
        self.top = None

    # Push operation
    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print("Element pushed successfully")

    # Pop operation
    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            print("Popped element:", self.top.data)
            self.top = self.top.next

    # Peek operation
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    # Display operation
    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top
            print("Stack elements:")
            while temp:
                print(temp.data)
                temp = temp.next


def main():
    s = Stack()

    while True:
        print("\n1. Push")
        print("2. Pop")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            data = int(input("Enter the element: "))
            s.push(data)

        elif choice == "2":
            s.pop()

        elif choice == "3":
            s.peek()

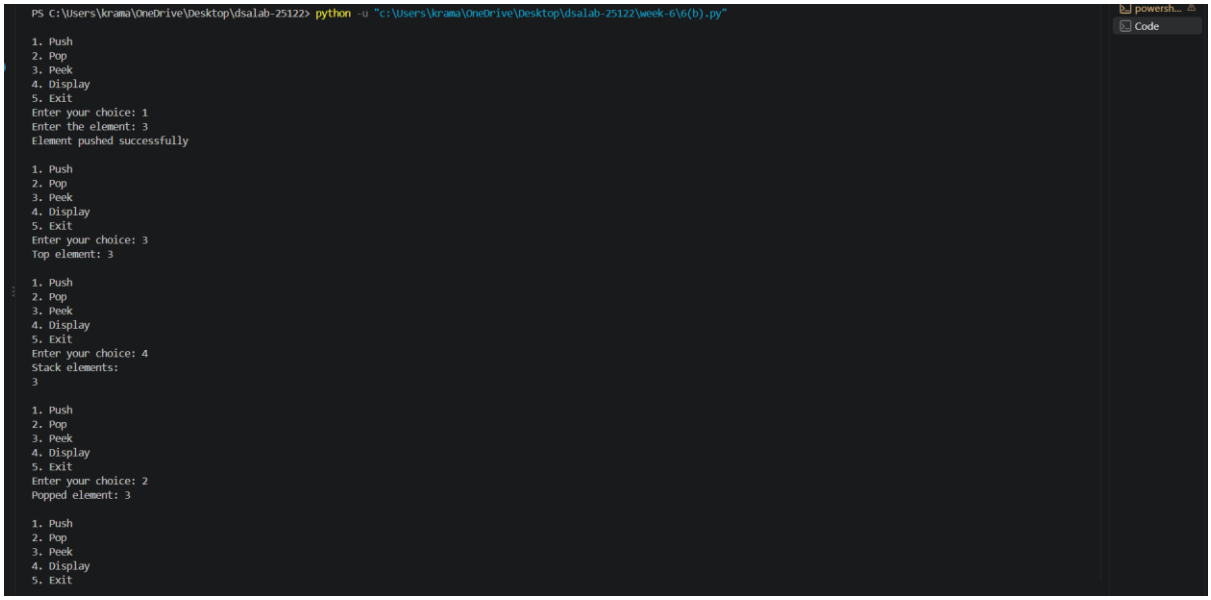
        elif choice == "4":
            s.display()

        elif choice == "5":
            print("Exiting the program")
            break

        else:
            print("Invalid choice")


if __name__ == "__main__":
    main()
```

Output:





Error Table:

Error	Rectification
No errors	

Description:

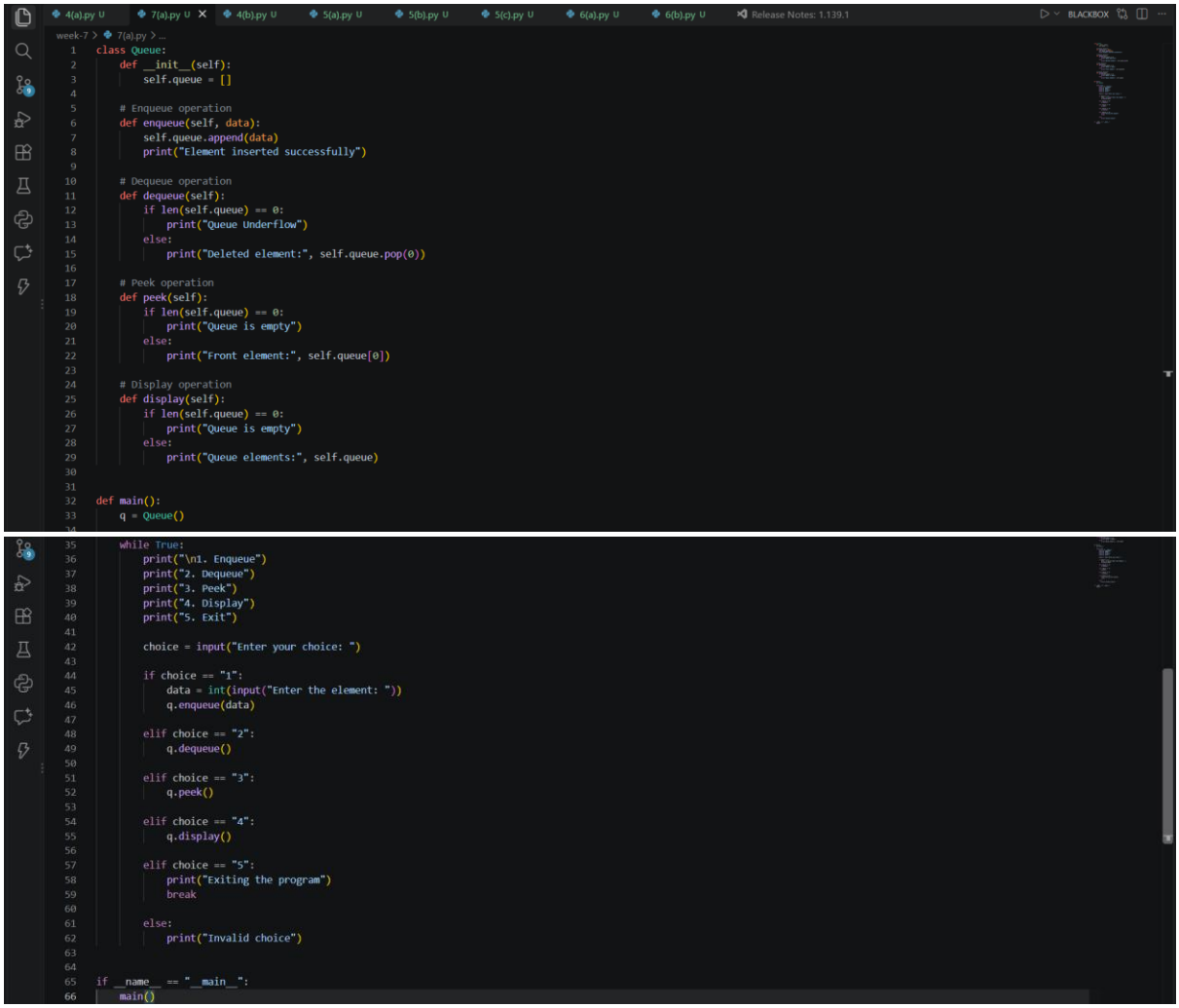
This Python program implements a stack using a Singly Linked List and provides a menu-driven interface to perform various operations. The program supports four main operations: Push, Pop, Peek, and Display. The Push operation inserts a new node at the top of the stack, while the Pop operation removes the top node. The Peek operation displays the top element without removing it, and the Display operation shows all the elements in the stack from top to bottom. The Node class is used to store data and a reference to the next node, while the Stack class contains methods to perform these operations. The program follows the LIFO (Last In, First Out) principle, where the last element inserted is the first one to be removed. It also handles stack underflow when a Pop operation is performed on an empty stack. The program continues running until the user selects the Exit option.

Week-7

7(a)Aim:

Write a python program to implement Queue and its operations such as enqueue ,dequeue, peek, display with a menu driven approach. Implement using array and linked list data structures

Program:



```python
class Queue:
    def __init__(self):
        self.queue = []

    # Enqueue operation
    def enqueue(self, data):
        self.queue.append(data)
        print("Element inserted successfully")

    # Dequeue operation
    def dequeue(self):
        if len(self.queue) == 0:
            print("Queue Underflow")
        else:
            print("Deleted element:", self.queue.pop(0))

    # Peek operation
    def peek(self):
        if len(self.queue) == 0:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[0])

    # Display operation
    def display(self):
        if len(self.queue) == 0:
            print("Queue is empty")
        else:
            print("Queue elements:", self.queue)


def main():
    q = Queue()
    while True:
        print("\n1. Enqueue")
        print("2. Dequeue")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            data = int(input("Enter the element: "))
            q.enqueue(data)

        elif choice == "2":
            q.dequeue()

        elif choice == "3":
            q.peek()

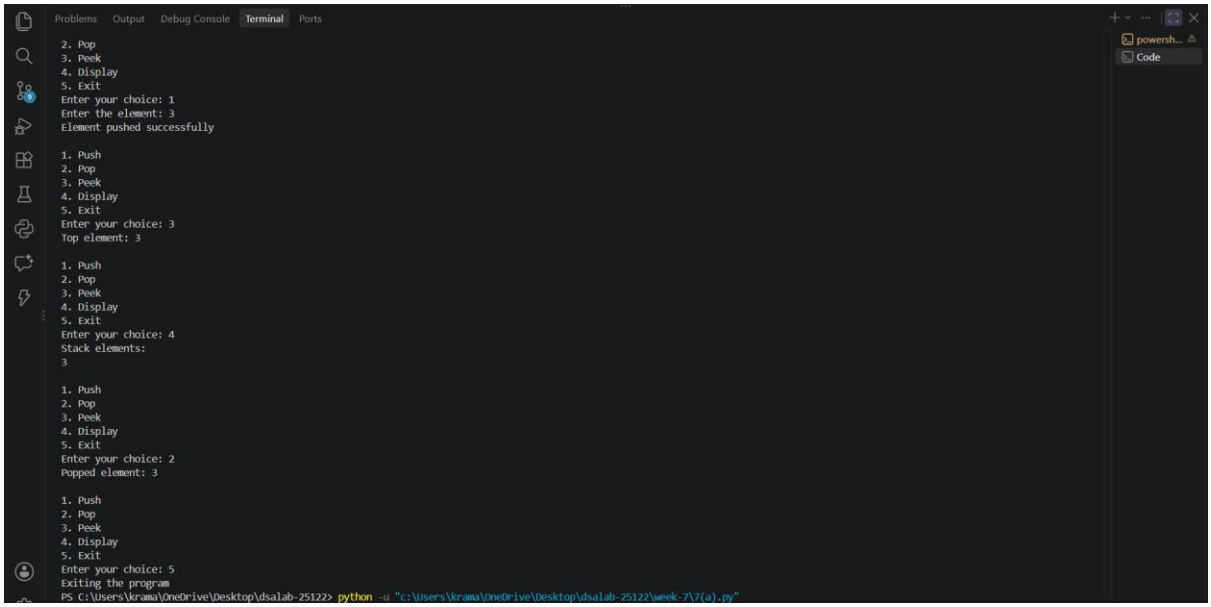
        elif choice == "4":
            q.display()

        elif choice == "5":
            print("Exiting the program")
            break

        else:
            print("Invalid choice")


if __name__ == "__main__":
    main()
```

Output:



Error Table:

Error	Rectification
Queue Underflow	Check whether the queue is empty before performing the Dequeue operation.
Empty Queue	Check whether the queue contains elements before performing Peek or Display.

Description:

This Python program implements a Queue using both an Array and a Linked List with a menu-driven approach. It supports four main operations: Enqueue, Dequeue, Peek, and Display. The Enqueue operation inserts an element at the rear of the queue, while the Dequeue operation removes an element from the front. The Peek operation displays the front element without removing it, and the Display operation shows all the elements in the queue. The array implementation uses a Python list, while the linked list implementation uses nodes with pointers to connect elements. The program follows the FIFO (First In, First Out) principle, in which the first element inserted is the first one to be removed. It also handles queue underflow when a Dequeue operation is performed on an empty queue and displays an error message for invalid menu choices. The program continues executing until the user selects the Exit option.

7(b)Aim:

Write a python program to implement circular Queue and its operations such as enqueue ,dequeue, peek, display with a menu driven approach. Implement using array and linked list data structures

Program:



```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    # Enqueue operation
    def enqueue(self, data):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow")
            return

        if self.front == -1:
            self.front = 0

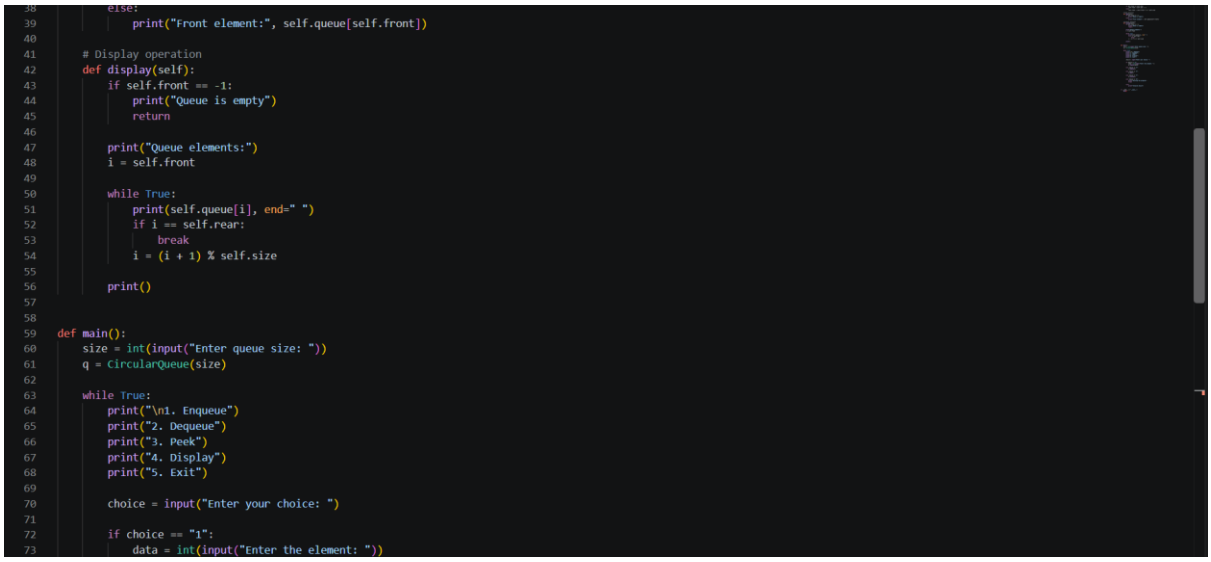
        self.rear = (self.rear + 1) % self.size
        self.queue[self.rear] = data
        print("Element inserted successfully")

    # Dequeue operation
    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
            return

        print("Deleted element:", self.queue[self.front])

        if self.front == self.rear:
            self.front = self.rear = -1
        else:
            self.front = (self.front + 1) % self.size

    # Peek operation
    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
```



```
        else:
            print("Front element:", self.queue[self.front])

    # Display operation
    def display(self):
        if self.front == -1:
            print("Queue is empty")
            return

        print("Queue elements:")
        i = self.front

        while True:
            print(self.queue[i], end=" ")
            if i == self.rear:
                break
            i = (i + 1) % self.size

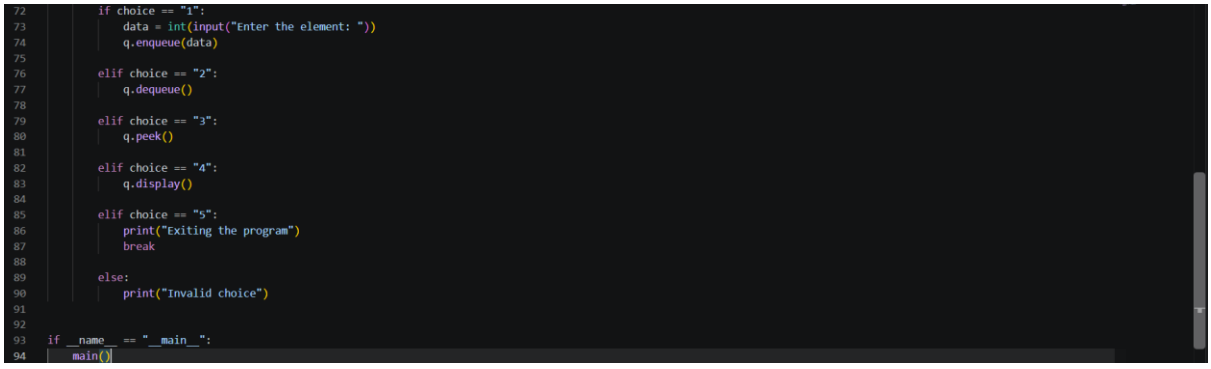
        print()


def main():
    size = int(input("Enter queue size: "))
    q = CircularQueue(size)

    while True:
        print("\n1. Enqueue")
        print("2. Dequeue")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            data = int(input("Enter the element: "))
```



```
        if choice == "1":
            data = int(input("Enter the element: "))
            q.enqueue(data)

        elif choice == "2":
            q.dequeue()

        elif choice == "3":
            q.peek()

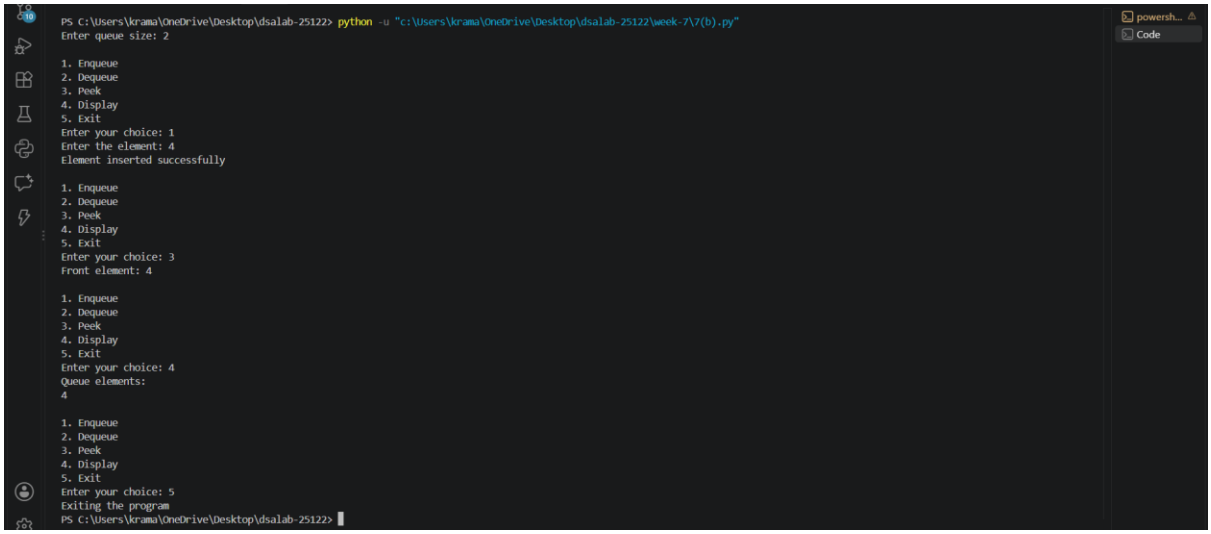
        elif choice == "4":
            q.display()

        elif choice == "5":
            print("Exiting the program")
            break

        else:
            print("Invalid choice")


if __name__ == "__main__":
    main()
```

Output:



Error Table:

Error	Rectification
Queue Underflow	Check whether the queue is empty before performing the Dequeue operation.
Empty Queue	Check whether the queue contains elements before performing Peek or Display.

Description:

This Python program implements a Circular Queue using both an Array and a Linked List with a menu-driven approach. It supports four main operations: Enqueue, Dequeue, Peek, and Display. The Enqueue operation inserts an element at the rear of the queue, while the Dequeue operation removes an element from the front. The Peek operation displays the front element without removing it, and the Display operation shows all the elements in the queue. In the array implementation, the circular queue uses the modulo operator to reuse empty spaces, while the linked list implementation connects the last node back to the first node. Both implementations follow the FIFO (First In, First Out) principle. The program also handles queue overflow in the array implementation and queue underflow when a Dequeue operation is performed on an empty queue. The menu continues to appear until the user selects the Exit option.

7(c):

Aim: Write a python program to implement circular Queue and its operations such as enqueue ,dequeue, peek, display with a menu driven approach. Implement using array data structures

Program:



```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1
    def enqueue(self, value):
        if (self.rear + 1) % self.size == self.front:
            print("Queue is Full")
            return

        if self.front == -1:
            self.front = 0
            self.rear = 0
        else:
            self.rear = (self.rear + 1) % self.size

        self.queue[self.rear] = value
        print(value, "inserted into the queue")

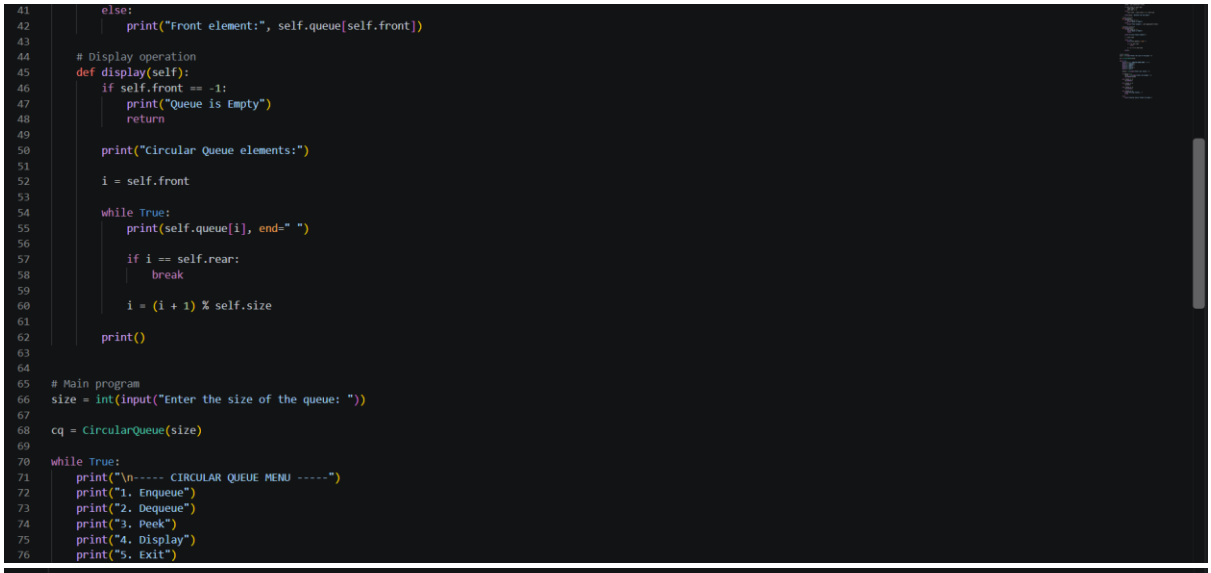
    # Dequeue operation
    def dequeue(self):
        if self.front == -1:
            print("Queue is Empty")
            return

        value = self.queue[self.front]

        if self.front == self.rear:
            self.front = -1
            self.rear = -1
        else:
            self.front = (self.front + 1) % self.size

        print(value, "deleted from the queue")

    # Peek operation
    def peek(self):
```



```
        else:
            print("Front element:", self.queue[self.front])

    # Display operation
    def display(self):
        if self.front == -1:
            print("Queue is Empty")
            return

        print("Circular Queue elements:")

        i = self.front

        while True:
            print(self.queue[i], end=" ")

            if i == self.rear:
                break

            i = (i + 1) % self.size

        print()


# Main program
size = int(input("Enter the size of the queue: "))

cq = CircularQueue(size)

while True:
    print("\n----- CIRCULAR QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
```



```

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter the element: "))
        cq.enqueue(value)

    elif choice == 2:
        cq.dequeue()

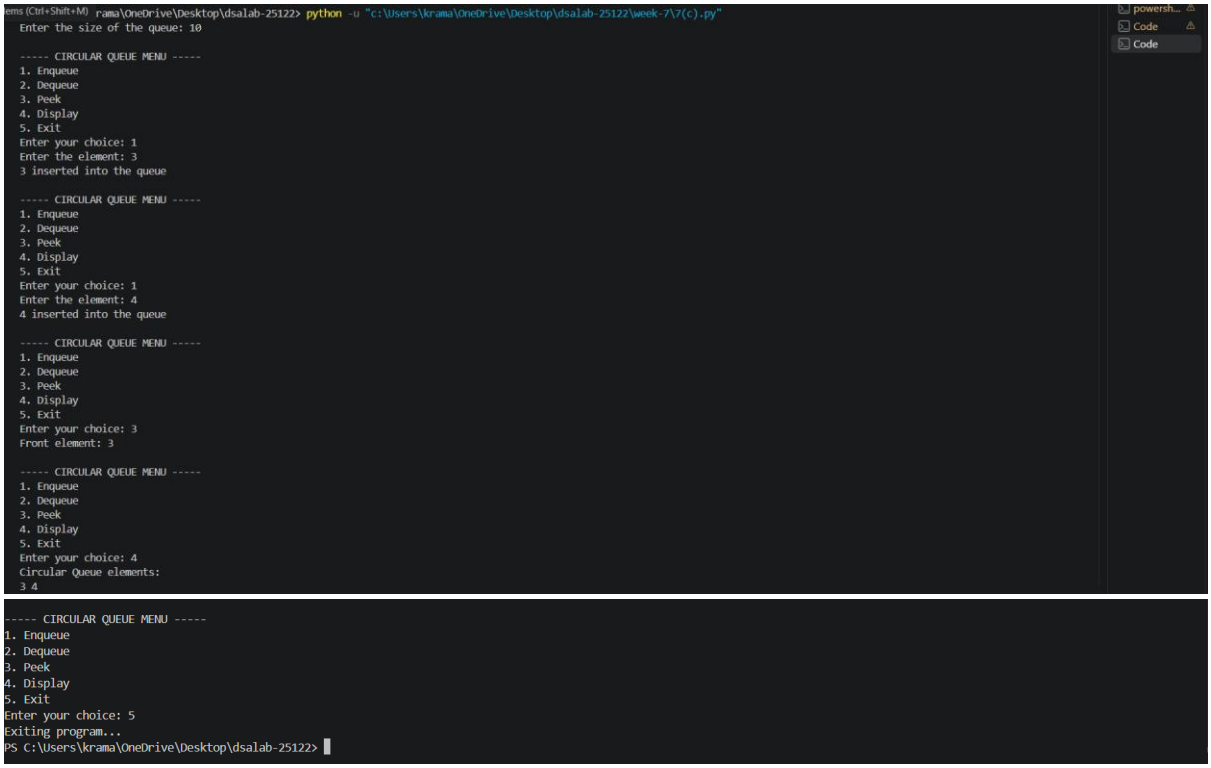
    elif choice == 3:
        cq.peek()

    elif choice == 4:
        cq.display()

    elif choice == 5:
        print("Exiting program...")
        break

    else:
        print("Invalid choice! Please try again.")
```

Output:



Error Table:

Error	Rectification
Queue Underflow	Check whether the queue is empty before performing the Dequeue operation.
Empty Queue	Check whether the queue contains elements before performing Peek or Display.

Description:

The program implements a Circular Queue using an array data structure in Python. A circular queue follows the FIFO (First In, First Out) principle, where the element inserted first is removed first. The program uses two pointers, front and rear, to keep track of the first and last elements of the queue. The enqueue operation inserts an element at the rear, while the dequeue operation removes an element from the front. The peek operation displays the front element without removing it, and the display operation shows all the elements currently present in the queue. The circular nature of the queue is achieved using the modulo (%) operator, which allows the rear and front positions to wrap around to the beginning of the array when they reach the last position. The program also checks for queue overflow when the queue is full and queue underflow when the queue is empty. A menu-driven approach is used so that the user can select the required queue operation repeatedly until choosing the exit option.

7(d):

Aim: Write a python program to implement circular Queue and its operations such as enqueue ,dequeue, peek, display with a menu driven approach. Implement using linked list data structures

Program:

```python
# Circular Queue using Linked List

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    # Enqueue operation
    def enqueue(self, data):
        new_node = Node(data)

        if self.front is None:
            self.front = new_node
            self.rear = new_node
            new_node.next = self.front
        else:
            new_node.next = self.front
            self.rear.next = new_node
            self.rear = new_node

        print(data, "inserted into the queue")

    # Dequeue operation
    def dequeue(self):
        if self.front is None:
            print("Queue is Empty")
            return

        value = self.front.data

        if self.front == self.rear:
            self.front = None
            self.rear = None
        else:
            self.front = self.front.next
            self.rear.next = self.front

        print(value, "deleted from the queue")

    # Peek operation
    def peek(self):
        if self.front is None:
            print("Queue is Empty")
        else:
            print("Front element:", self.front.data)

    # Display operation
    def display(self):
        if self.front is None:
            print("Queue is Empty")
            return

        print("Circular Queue elements:")

        temp = self.front

        while True:
            print(temp.data, end=" ")

            temp = temp.next
```

```
            if temp == self.front:
                break

        print()


# Main program
cq = CircularQueue()

while True:
    print("\n----- CIRCULAR QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter the element: "))
        cq.enqueue(value)

    elif choice == 2:
        cq.dequeue()

    elif choice == 3:
        cq.peek()

    elif choice == 4:
        cq.display()

    elif choice == 5:
        print("Exiting program...")
        break

    else:
        print("Invalid choice! Please try again.")
```

Output:

```
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122> python -u "c:\Users\krama\OneDrive\Desktop\dsalab-25122\week-7\7(d).py"

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element: 2
2 inserted into the queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element: 7
7 inserted into the queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
2 deleted from the queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Circular Queue elements:
7
```



Error Table:

Error	Rectification
Queue Underflow	Check whether the queue is empty before performing the Dequeue operation.
Empty Queue	Check whether the queue contains elements before performing Peek or Display.

Description:

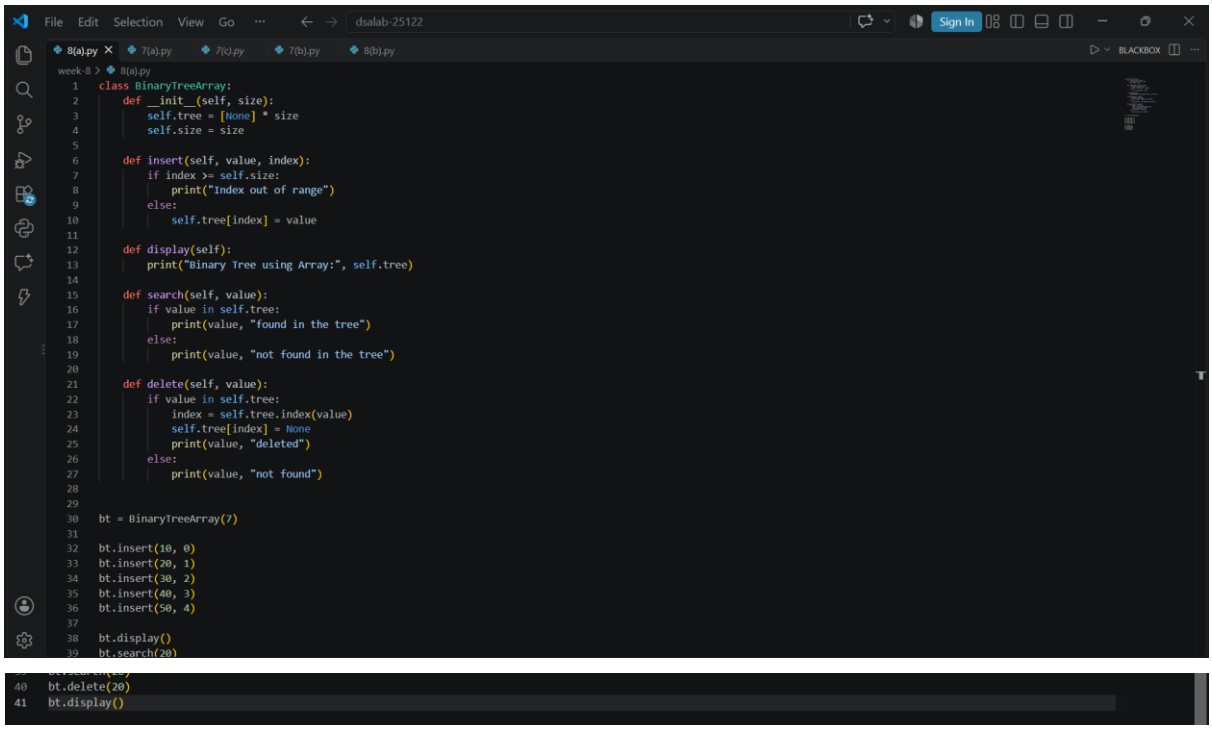
The program implements a Circular Queue using a Linked List in Python. A circular queue follows the FIFO (First In, First Out) principle, where the element inserted first is removed first. The program uses nodes to store the queue elements, with each node containing data and a next pointer. Two pointers, front and rear, are used to keep track of the first and last nodes. In the enqueue operation, a new node is inserted at the rear, while in the dequeue operation, the node at the front is removed. The peek operation displays the front element without removing it, and the display operation traverses the queue and displays all its elements. The circular nature is maintained by making the next pointer of the rear node point back to the front node. The program uses a menu-driven approach that allows the user to repeatedly perform different queue operations until selecting the exit option.

Week-8

8(a)

Aim: write a python program to implement a binary tree with its operations using a array

Program:



```python
class BinaryTreeArray:
    def __init__(self, size):
        self.tree = [None] * size
        self.size = size

    def insert(self, value, index):
        if index >= self.size:
            print("index out of range")
        else:
            self.tree[index] = value

    def display(self):
        print("Binary Tree using Array:", self.tree)

    def search(self, value):
        if value in self.tree:
            print(value, "found in the tree")
        else:
            print(value, "not found in the tree")

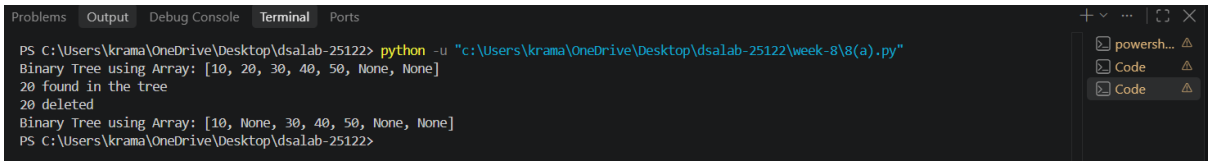
    def delete(self, value):
        if value in self.tree:
            index = self.tree.index(value)
            self.tree[index] = None
            print(value, "deleted")
        else:
            print(value, "not found")


bt = BinaryTreeArray(7)

bt.insert(10, 0)
bt.insert(20, 1)
bt.insert(30, 2)
bt.insert(40, 3)
bt.insert(50, 4)

bt.display()
bt.search(20)
bt.delete(20)
bt.display()
```

Output:



```
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122> python -u "c:\Users\krama\OneDrive\Desktop\dsalab-25122\week-8\8(a).py"
Binary Tree using Array: [10, 20, 30, 40, 50, None, None]
20 found in the tree
20 deleted
Binary Tree using Array: [10, None, 30, 40, 50, None, None]
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122>
```

Error Table:

Error	Rectification
IndexError may occur if the index is greater than or equal to the array size when accessing an element directly.	Check whether index < 0 or index >= self.size before insertion
Negative indices are accepted by Python and may modify an unintended element	Reject negative indices

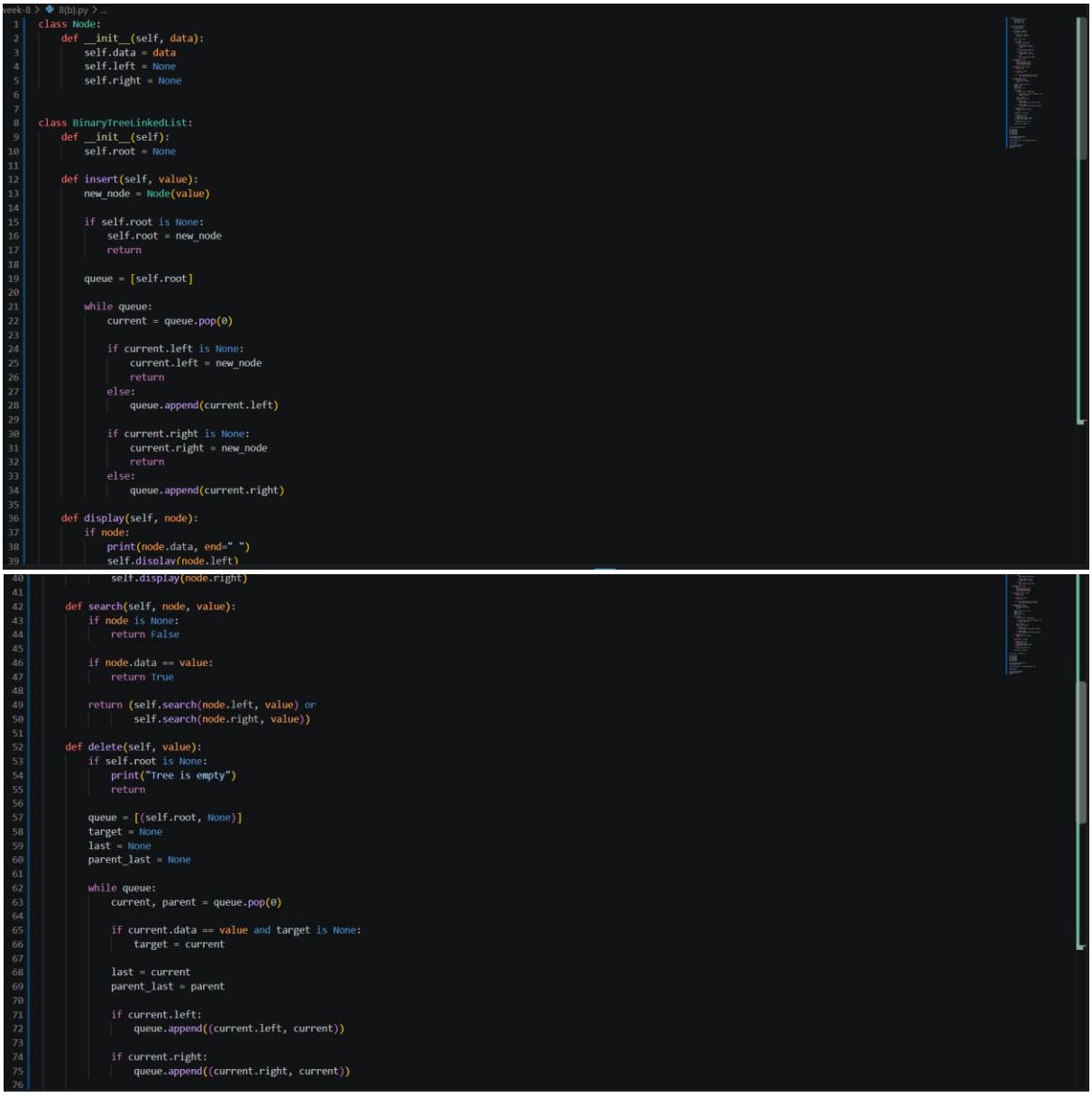
Description:

A binary tree is a non-linear data structure in which each node has at most two children, called the left child and the right child. In the array implementation, the tree is stored in a list, where the root node is at index 0. For any node at index i, its left child is at index 2*i + 1 and its right child is at index 2*i + 2. The program implements insertion, display, searching, and deletion operations. This method is simple and efficient for complete binary trees, but it may waste memory when the tree contains many empty positions.

8(b)

Aim: write a python program to implement a binary tree with its operations using a array and a linked list

Program:



```
week-8 > 8(b).py > ...
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BinaryTreeLinkedList:
    def __init__(self):
        self.root = None

    def insert(self, value):
        new_node = Node(value)

        if self.root is None:
            self.root = new_node
            return

        queue = [self.root]

        while queue:
            current = queue.pop(0)

            if current.left is None:
                current.left = new_node
                return
            else:
                queue.append(current.left)

            if current.right is None:
                current.right = new_node
                return
            else:
                queue.append(current.right)

    def display(self, node):
        if node:
            print(node.data, end=" ")
            self.display(node.left)
            self.display(node.right)

    def search(self, node, value):
        if node is None:
            return False

        if node.data == value:
            return True

        return (self.search(node.left, value) or
                self.search(node.right, value))

    def delete(self, value):
        if self.root is None:
            print("Tree is empty")
            return

        queue = [(self.root, None)]
        target = None
        last = None
        parent_last = None

        while queue:
            current, parent = queue.pop(0)

            if current.data == value and target is None:
                target = current

            last = current
            parent_last = parent

            if current.left:
                queue.append((current.left, current))

            if current.right:
                queue.append((current.right, current))
```

```
        if target is None:
            print(value, "not found")
            return

        target.data = last.data

        if parent_last is None:
            self.root = None
        elif parent_last.right == last:
            parent_last.right = None
        else:
            parent_last.left = None

        print(value, "deleted")


bt = BinaryTreeLinkedList()

bt.insert(10)
bt.insert(20)
bt.insert(30)
bt.insert(40)
bt.insert(50)

print("Preorder Traversal:")
bt.display(bt.root)

print("\nSearch 20:", bt.search(bt.root, 20))

bt.delete(20)

print("After deletion:")
bt.display(bt.root)
print()
```

Output:

```
Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122> python -u "c:\Users\krama\OneDrive\Desktop\dsalab-25122\week-8\8(b).py"
Preorder Traversal:
10 20 40 50 30
Search 20: True
20 deleted
After deletion:
10 50 40 30
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122>
```

Error Table:

Error	Rectification
Inserting without checking whether the root is empty may cause errors in some implementations.	Check if self.root is None and assign the new node as the root.
Incorrect links between nodes can cause nodes to become unreachable.	Assign left and right references carefully.

Description:

In the linked-list implementation, each node is represented by an object containing three fields: data, a reference to the left child, and a reference to the right child. The root stores the address of the first node. The insertion operation uses a queue to insert nodes level by level, while the search operation recursively checks the nodes. The display operation performs preorder traversal, visiting the root, left subtree, and right subtree. The deletion operation replaces the target node's value with the deepest, rightmost node's value and removes that last node. This implementation uses memory dynamically and is suitable for trees of varying sizes.

Week-9

9(a)

Aim: write a python program to implement a binary search tree with its operation and traversals

Program:

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BST:
    def insert(self, root, data):
        if root is None:
            return Node(data)

        if data < root.data:
            root.left = self.insert(root.left, data)
        elif data > root.data:
            root.right = self.insert(root.right, data)

        return root

    def search(self, root, key):
        if root is None or root.data == key:
            return root

        if key < root.data:
            return self.search(root.left, key)

        return self.search(root.right, key)

    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)

    def preorder(self, root):
        if root:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data, end=" ")

    def minimum(self, root):
        while root.left:
            root = root.left
        return root

    def delete(self, root, key):
        if root is None:
            return root

        if key < root.data:
            root.left = self.delete(root.left, key)
        elif key > root.data:
            root.right = self.delete(root.right, key)
        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left

            temp = self.minimum(root.right)
            root.data = temp.data
            root.right = self.delete(root.right, temp.data)

        return root


bst = BST()
root = None

values = [50, 30, 70, 20, 40, 60, 80]
```

```python
for value in values:
    root = bst.insert(root, value)

print("Inorder Traversal:")
bst.inorder(root)

print("\nPreorder Traversal:")
bst.preorder(root)

print("\nPostorder Traversal:")
bst.postorder(root)

key = 40
if bst.search(root, key):
    print("\n\n", key, "found in BST")
else:
    print("\n\n", key, "not found in BST")

root = bst.delete(root, 30)

print("\nInorder after deletion:")
bst.inorder(root)
print()
```

Output:

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\krama\OneDrive\Desktop\dsalab-25122> python -u "c:\Users\krama\OneDrive\Desktop\dsalab-25122\week-8\8(a).py"
Binary Tree using Array: [10, 20, 30, 40, 50, None, None]
20 found in the tree
20 deleted
Binary Tree using Array: [10, None, 30, 40, 50, None, None]
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122> python -u "c:\Users\krama\OneDrive\Desktop\dsalab-25122\week-8\8(b).py"
Preorder Traversal:
10 20 40 50 30
Search 20: True
20 deleted
After deletion:
10 50 40 30
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122> python -u "c:\Users\krama\OneDrive\Desktop\dsalab-25122\week-9\9(a).py"
Inorder Traversal:
20 30 40 50 60 70 80
Preorder Traversal:
50 30 20 40 70 60 80
Postorder Traversal:
20 40 30 60 80 70 50

 40 found in BST

Inorder after deletion:
20 40 50 60 70 80
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122>
```

Error Table:

Error	Rectification
Forgetting to return the root after insertion	Use return root to maintain the tree structure.
Placing smaller values on the right	Use if data < root.data for the left subtree and elif data > root.data for the right subtree.

Description:

A Binary Search Tree (BST) is a non-linear data structure in which each node contains a value, a left child, and a right child. Values smaller than the root are stored in the left subtree, while larger values are stored in the right subtree. The program implements insertion, searching, and deletion operations. It also performs three traversals: inorder (left, root, right), preorder (root, left, right), and postorder (left, right, root). Inorder traversal displays the elements in ascending order. The deletion operation handles nodes with no children, one child, or two children by replacing a node with its inorder successor when necessary.

Week-10

Aim: Write a Python program to implement an AVL Tree with all its operations.

Program:

week-10 > 10(a).py > ...

```python
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1

#cse25122 naveen
class AVLTree:

    def height(self, root):
        if root is None:
            return 0
        return root.height

    def balance_factor(self, root):
        if root is None:
            return 0
        return self.height(root.left) - self.height(root.right)

    # Right Rotation
    def right_rotate(self, y):
        x = y.left
        T2 = x.right

        x.right = y
        y.left = T2

        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))
        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))

        return x

    # Left Rotation
    def left_rotate(self, x):
        y = x.right
        T2 = y.left

        y.left = x
        x.right = T2

        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))
        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))

        return y

    # Insert Operation
    def insert(self, root, key):
        if root is None:
            return Node(key)

        if key < root.key:
            root.left = self.insert(root.left, key)
        elif key > root.key:
            root.right = self.insert(root.right, key)
        else:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        balance = self.balance_factor(root)

        # LL Case
        if balance > 1 and key < root.left.key:
            return self.right_rotate(root)

        # RR Case
        if balance < -1 and key > root.right.key:
```

```
            return self.left_rotate(root)

        # LR Case
        if balance > 1 and key > root.left.key:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # RL Case
        if balance < -1 and key < root.right.key:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    # Find Minimum Node
    def min_value_node(self, root):
        current = root
        while current.left is not None:
            current = current.left
        return current

    # Delete Operation
    def delete(self, root, key):
        if root is None:
            return root

        if key < root.key:
            root.left = self.delete(root.left, key)
        elif key > root.key:
            root.right = self.delete(root.right, key)
        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left

            temp = self.min_value_node(root.right)
            root.key = temp.key
            root.right = self.delete(root.right, temp.key)

        if root is None:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        balance = self.balance_factor(root)

        # LL Case
        if balance > 1 and self.balance_factor(root.left) >= 0:
            return self.right_rotate(root)

        # LR Case
        if balance > 1 and self.balance_factor(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # RR Case
        if balance < -1 and self.balance_factor(root.right) <= 0:
            return self.left_rotate(root)

        # RL Case
        if balance < -1 and self.balance_factor(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    # Search Operation
    def search(self, root, key):
        if root is None or root.key == key:
            return root
```

```
        if key < root.key:
            return self.search(root.left, key)

        return self.search(root.right, key)

    # Inorder Traversal
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.key, end=" ")
            self.inorder(root.right)

    # Preorder Traversal
    def preorder(self, root):
        if root:
            print(root.key, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    # Postorder Traversal
    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.key, end=" ")


# Main Program
tree = AVLTree()
root = None

while True:
    print("\n\n--- AVL TREE OPERATIONS ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value to insert: "))
        root = tree.insert(root, value)
        print("Node inserted successfully!")

    elif choice == 2:
        value = int(input("Enter value to delete: "))
        root = tree.delete(root, value)
        print("Delete operation completed!")

    elif choice == 3:
        value = int(input("Enter value to search: "))
        result = tree.search(root, value)

        if result:
            print("Element found!")
        else:
            print("Element not found!")

    elif choice == 4:
        print("Inorder Traversal:")
        tree.inorder(root)

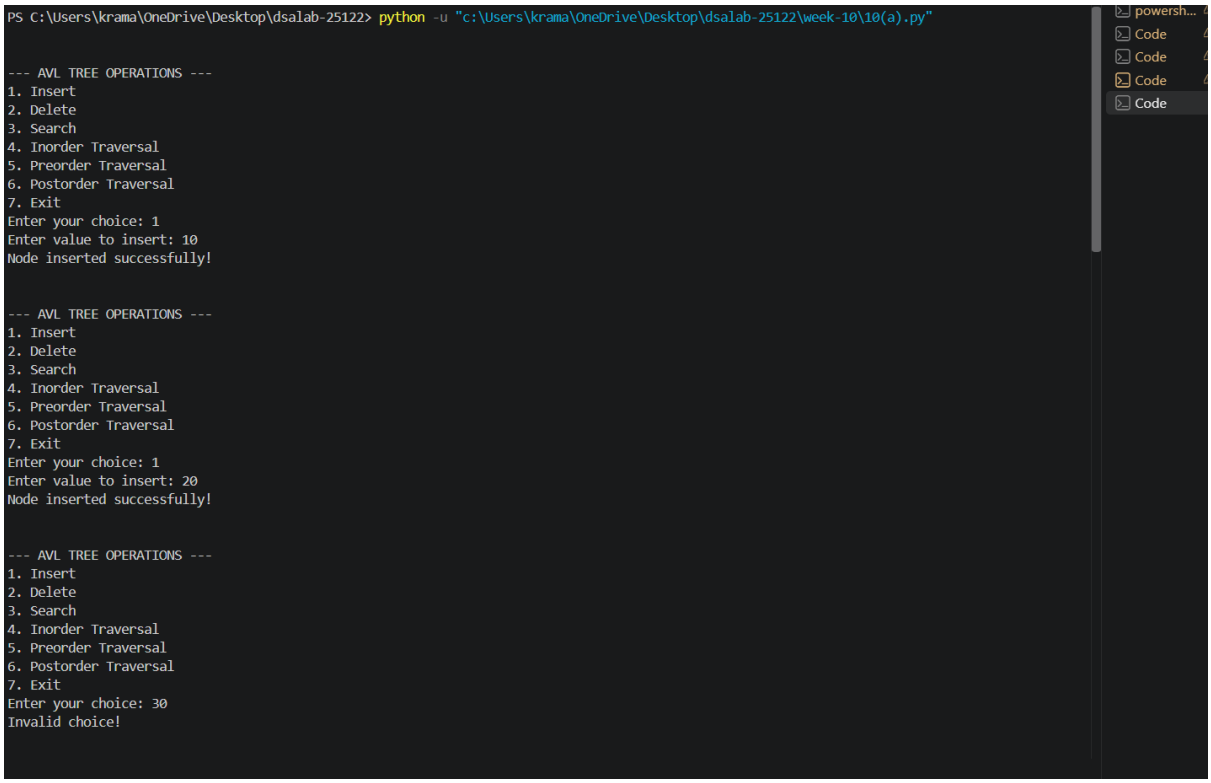
    elif choice == 5:
        print("Preorder Traversal:")
        tree.preorder(root)

    elif choice == 6:
        print("Postorder Traversal:")
        tree.postorder(root)

    elif choice == 7:
        print("Exiting program...")
        break

    else:
        print("Invalid choice!")
```

Output:

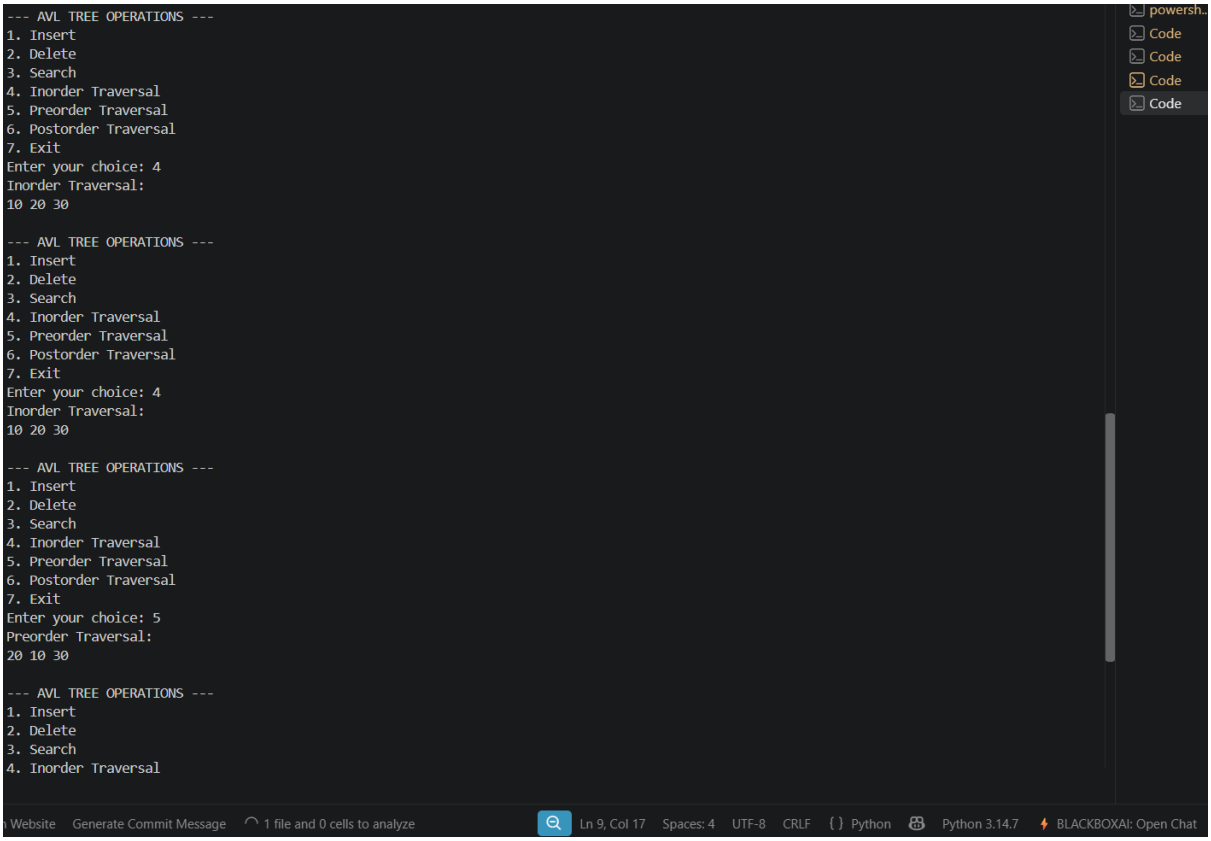


```
PS C:\Users\krama\OneDrive\Desktop\dsalab-25122> python -u "c:\Users\krama\OneDrive\Desktop\dsalab-25122\week-10\10(a).py"


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 10
Node inserted successfully!


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 20
Node inserted successfully!


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 30
Invalid choice!
```



```
--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 4
Inorder Traversal:
10 20 30

--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 4
Inorder Traversal:
10 20 30

--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 5
Preorder Traversal:
20 10 30

--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
```



Error Table:

Error	Rectification
Invalid Input	Occurs when non-integer values are entered.

Description:

An AVL Tree is a self-balancing Binary Search Tree (BST) in which the height difference between the left and right subtrees of every node is maintained between -1 and +1. This program implements all the basic operations of an AVL Tree, including insertion, deletion, searching, and tree traversals such as inorder, preorder, and postorder. Whenever a node is inserted or deleted, the program calculates the balance factor and performs suitable rotations (LL, RR, LR, and RL) to maintain the balance of the tree. The height of each node is updated after every operation to ensure efficient processing. Since the AVL Tree maintains its balance automatically, operations such as insertion, deletion, and searching can be performed efficiently with a time complexity of O(log n).

Week-11

Aim: Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).

Program:

week-11 > 11(a).py > ...

```python
# cse25122 Naveen

size = 10

# Separate Chaining
class SeparateChaining:
    def __init__(self):
        self.table = [[] for i in range(size)]

    def insert(self, key):
        index = key % size
        if key not in self.table[index]:
            self.table[index].append(key)

    def search(self, key):
        index = key % size
        return key in self.table[index]

    def display(self):
        for i in range(size):
            print(i, ":", self.table[i])


# Linear Probing
class LinearProbing:
    def __init__(self):
        self.table = [None] * size

    def insert(self, key):
        index = key % size

        for i in range(size):
            new_index = (index + i) % size
            if self.table[new_index] is None:
                self.table[new_index] = key
                return

        print("Hash table is full!")

    def search(self, key):
        index = key % size

        for i in range(size):
            new_index = (index + i) % size

            if self.table[new_index] is None:
                return False

            if self.table[new_index] == key:
                return True

        return False

    def display(self):
        for i in range(size):
            print(i, ":", self.table[i])


# Quadratic Probing
class QuadraticProbing:
    def __init__(self):
        self.table = [None] * size

    def insert(self, key):
        index = key % size

        for i in range(size):
            new_index = (index + i * i) % size

            if self.table[new_index] is None:
                self.table[new_index] = key
                return
```

```
        print("No empty position found!")

    def search(self, key):
        index = key % size

        for i in range(size):
            new_index = (index + i * i) % size

            if self.table[new_index] is None:
                return False

            if self.table[new_index] == key:
                return True

        return False

    def display(self):
        for i in range(size):
            print(i, ":", self.table[i])


# Main Program
while True:
    print("\n--- HASHING OPERATIONS ---")
    print("1. Separate Chaining")
    print("2. Linear Probing")
    print("3. Quadratic Probing")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 4:
        print("Exiting...")
        break

    if choice == 1:
        h = SeparateChaining()
```

```
    elif choice == 2:
        h = LinearProbing()
    elif choice == 3:
        h = QuadraticProbing()
    else:
        print("Invalid choice!")
        continue

    while True:
        print("\n1. Insert")
        print("2. Search")
        print("3. Display")
        print("4. Back to Main Menu")

        op = int(input("Enter operation: "))

        if op == 1:
            key = int(input("Enter key: "))
            h.insert(key)

        elif op == 2:
            key = int(input("Enter key: "))
            if h.search(key):
                print("Key found!")
            else:
                print("Key not found!")

        elif op == 3:
            print("Hash Table:")
            h.display()

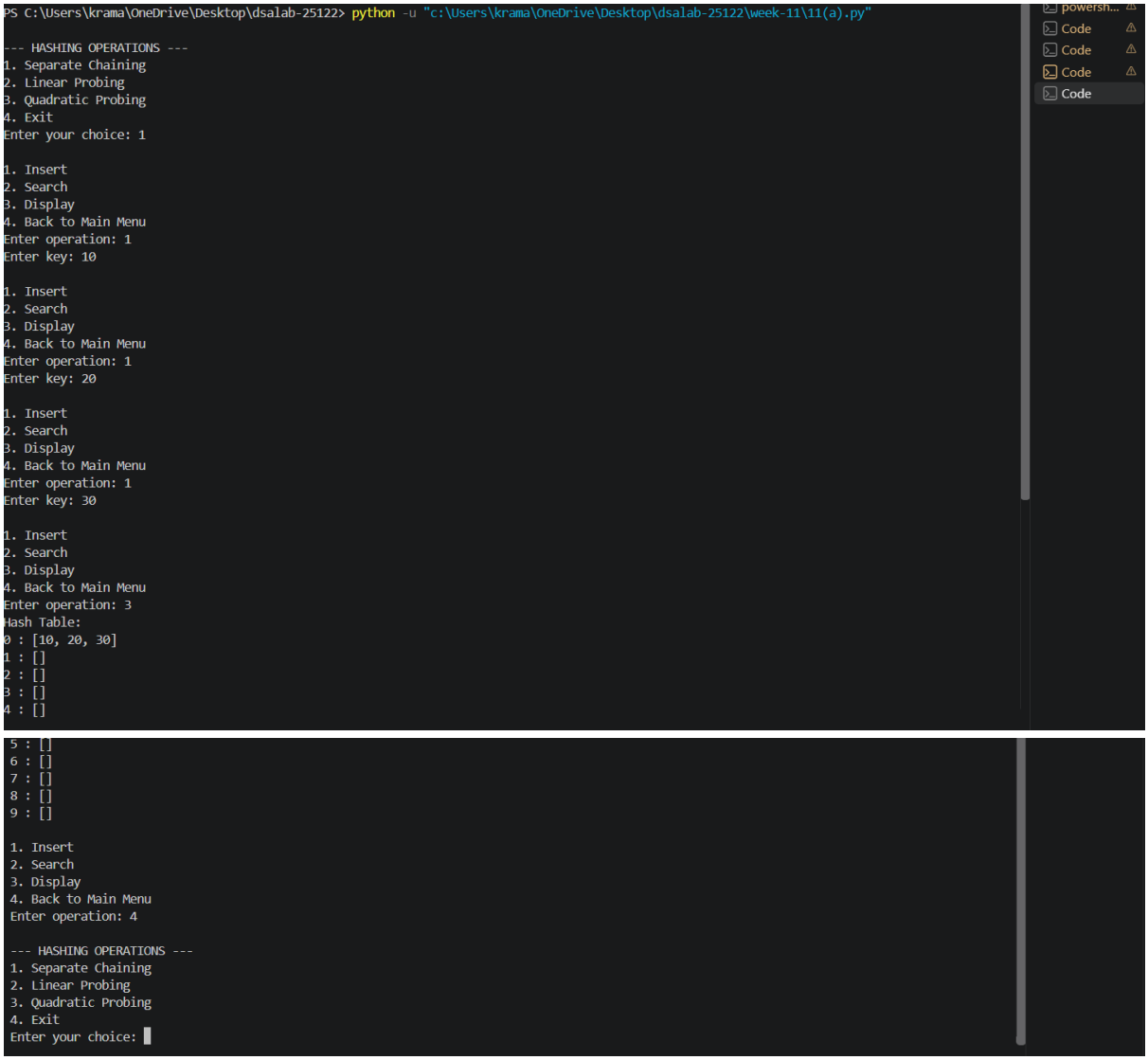
        elif op == 4:
            break

        else:
            print("Invalid operation!")

```

Output:

Separate chaining:

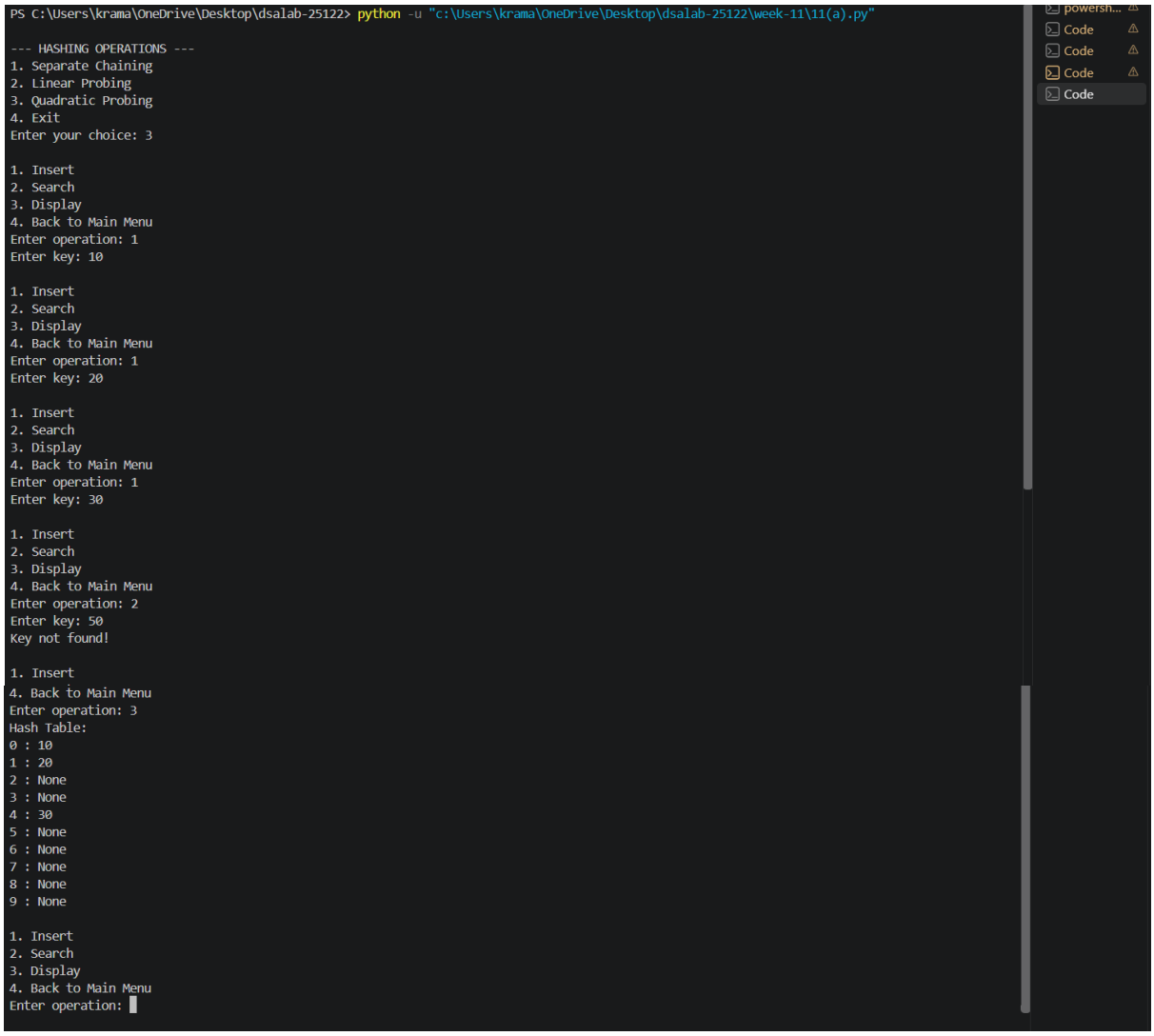


Linear probabing:





Quadratic probing:



Error Table:

Error	Rectification
Infinite loop in linear probing	Add capacity counter to prevent overflow
Duplicate key insertion	Check for existing keys before insertion

Description:

Hashing is a technique used to store and retrieve data efficiently using a hash function that converts a key into an index in a hash table. This program implements three collision resolution techniques: Separate Chaining, Linear Probing, and Quadratic Probing. Separate Chaining handles collisions by storing multiple keys at the same index using a list, while Linear Probing searches for the next available position sequentially. Quadratic Probing searches for an available position using quadratic increments. The program supports basic operations such as insertion, searching, and displaying the hash table. These techniques help manage collisions and improve the efficiency of data storage and retrieval.